

Demostraciones de algoritmos de fuerza bruta

Semántica, inducción, correctitud de podas y complejidad

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2026

Agenda de hoy

- 1 ¿Qué significa que un algoritmo de backtracking es correcto?
- 2 La semántica de una función recursiva.
- 3 Demostrar la recurrencia: inducción en lo que decrece.
- 4 Demostrar las podas: factibilidad y optimalidad (con estado global).
- 5 Problema completo 1: conjunto independiente de tamaño k .
- 6 Problema completo 2: cambio mínimo.
- 7 Complejidad, con un poco más de rigor.
- 8 Errores comunes.
- 9 Tips + Conclusiones.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.
- 2 **Parciales**, **sucesoras**, y el *resumen* de las decisiones pasadas.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.
- 2 **Parciales, sucesoras**, y el *resumen* de las decisiones pasadas.
- 3 **Función recursiva** con su **semántica**.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.
- 2 **Parciales**, **sucesoras**, y el *resumen* de las decisiones pasadas.
- 3 **Función recursiva** con su **semántica**.
- 4 **Podas** por factibilidad y optimalidad.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.
- 2 **Parciales**, **sucesoras**, y el *resumen* de las decisiones pasadas.
- 3 **Función recursiva** con su **semántica**.
- 4 **Podas** por factibilidad y optimalidad.
- 5 **Complejidad**.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.
- 2 **Parciales, sucesoras**, y el *resumen* de las decisiones pasadas.
- 3 **Función recursiva** con su **semántica**.
- 4 **Podas** por factibilidad y optimalidad.
- 5 **Complejidad**.
- 6 **Implementar** y testear.

La receta, otra vez

La clase pasada, frente a un enunciado:

- 1 **Candidatas** Sols y **válidas** válida(a); si es optimización, valor(a) y el orden.
- 2 **Parciales**, **sucesoras**, y el *resumen* de las decisiones pasadas.
- 3 **Función recursiva** con su **semántica**.
- 4 **Podas** por factibilidad y optimalidad.
- 5 **Complejidad**.
- 6 **Implementar** y testear.

Hoy: cómo se **demuestra** que los pasos 3 y 4 están bien, y con qué rigor se justifica el 5.
La clase de repaso de demostraciones vuelve, aplicada a algoritmos recursivos.

¿Qué significa que un algoritmo de backtracking es correcto?

Hay tres afirmaciones distintas escondidas en «el algoritmo es correcto»:

¿Qué significa que un algoritmo de backtracking es correcto?

Hay tres afirmaciones distintas escondidas en «el algoritmo es correcto»:

- 1 **La formalización es fiel:** $Sols_{válidas}$ representa exactamente las soluciones del enunciado, y $valor$ mide lo que el enunciado quiere maximizar. Esto se *argumenta* (a veces se demuestra: «una reina por fila y columna»).

¿Qué significa que un algoritmo de backtracking es correcto?

Hay tres afirmaciones distintas escondidas en «el algoritmo es correcto»:

- 1 **La formalización es fiel:** $Sols_{válidas}$ representa exactamente las soluciones del enunciado, y $valor$ mide lo que el enunciado quiere maximizar. Esto se *argumenta* (a veces se demuestra: «una reina por fila y columna»).
- 2 **La función recursiva computa lo que dice su semántica.** Esto se **demuestra por inducción**. Es el corazón de la clase de hoy.

¿Qué significa que un algoritmo de backtracking es correcto?

Hay tres afirmaciones distintas escondidas en «el algoritmo es correcto»:

- 1 **La formalización es fiel:** $Sols_{válidas}$ representa exactamente las soluciones del enunciado, y $valor$ mide lo que el enunciado quiere maximizar. Esto se *argumenta* (a veces se demuestra: «una reina por fila y columna»).
- 2 **La función recursiva computa lo que dice su semántica.** Esto se **demuestra por inducción**. Es el corazón de la clase de hoy.
- 3 **Las podas no cambian el resultado.** Esto se demuestra: *si $\neg R(p)$, ninguna extensión es válida; $U(p)$ acota a toda extensión válida.*

¿Qué significa que un algoritmo de backtracking es correcto?

Hay tres afirmaciones distintas escondidas en «el algoritmo es correcto»:

- 1 **La formalización es fiel:** $Sols_{válidas}$ representa exactamente las soluciones del enunciado, y $valor$ mide lo que el enunciado quiere maximizar. Esto se *argumenta* (a veces se demuestra: «una reina por fila y columna»).
 - 2 **La función recursiva computa lo que dice su semántica.** Esto se **demuestra por inducción**. Es el corazón de la clase de hoy.
 - 3 **Las podas no cambian el resultado.** Esto se demuestra: *si $\neg R(p)$, ninguna extensión es válida; $U(p)$ acota a toda extensión válida.*
- Y una cuarta que **no** se demuestra en esta materia: que el código implementa la fórmula. Se asume que el pasaje es directo (si no lo es, el problema es del código).

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.
- La función recursiva escrita como **fórmula matemática** (casos base y recursivo), no sólo como pseudocódigo.

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.
- La función recursiva escrita como **fórmula matemática** (casos base y recursivo), no sólo como pseudocódigo.
- Su **semántica**: una fórmula que dice qué valor computa para *cualquier* valor de los parámetros.

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.
- La función recursiva escrita como **fórmula matemática** (casos base y recursivo), no sólo como pseudocódigo.
- Su **semántica**: una fórmula que dice qué valor computa para *cualquier* valor de los parámetros.
- Una **demostración por inducción** de que la función cumple su semántica, y una observación de que la llamada inicial es el problema pedido.

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.
- La función recursiva escrita como **fórmula matemática** (casos base y recursivo), no sólo como pseudocódigo.
- Su **semántica**: una fórmula que dice qué valor computa para *cualquier* valor de los parámetros.
- Una **demostración por inducción** de que la función cumple su semántica, y una observación de que la llamada inicial es el problema pedido.
- Para cada poda: enunciado de la regla y demostración.

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.
- La función recursiva escrita como **fórmula matemática** (casos base y recursivo), no sólo como pseudocódigo.
- Su **semántica**: una fórmula que dice qué valor computa para *cualquier* valor de los parámetros.
- Una **demostración por inducción** de que la función cumple su semántica, y una observación de que la llamada inicial es el problema pedido.
- Para cada poda: enunciado de la regla y demostración.
- Complejidad: nodos $\times$ trabajo por nodo, y espacio.

Cómo se ve una respuesta completa

Para un ejercicio típico de la práctica («diseñar un algoritmo de backtracking, demostrar que es correcto, proponer una poda, calcular la complejidad»), una respuesta completa tiene:

- Las definiciones: Sols, válida, (opcional) valor; qué es una parcial y cuáles son sus sucesoras; qué parámetros recibe la función y qué representan.
- La función recursiva escrita como **fórmula matemática** (casos base y recursivo), no sólo como pseudocódigo.
- Su **semántica**: una fórmula que dice qué valor computa para *cualquier* valor de los parámetros.
- Una **demostración por inducción** de que la función cumple su semántica, y una observación de que la llamada inicial es el problema pedido.
- Para cada poda: enunciado de la regla y demostración.
- Complejidad: nodos $\times$ trabajo por nodo, y espacio.

El pseudocódigo es lo último y lo menos importante. Es la fórmula lo que se demuestra.

El ejemplo de siempre: mochila

n objetos, pesos $w_j \in \mathbb{N}$, valores $v_j \in \mathbb{N}$, capacidad W . Sols = $\{0, 1\}^n$,
 $\text{válida}(a) := [\sum_j a_j w_j \leq W]$, $\text{valor}(a) := \sum_j a_j v_j$.

```
function mochila( $i$ ,  $pa$ ,  $va$ )  
  if  $i = n + 1$  then return ( $pa \leq W$ ) ?  $va$  :  $-\infty$ ;  
  return  $\text{máx}(\text{mochila}(i + 1, pa, va), \text{mochila}(i + 1, pa + w_i, va + v_i))$  ;
```

El ejemplo de siempre: mochila

n objetos, pesos $w_j \in \mathbb{N}$, valores $v_j \in \mathbb{N}$, capacidad W . Sols = $\{0, 1\}^n$,
 $\text{válida}(a) := [\sum_j a_j w_j \leq W]$, $\text{valor}(a) := \sum_j a_j v_j$.

```
function mochila( $i$ ,  $pa$ ,  $va$ )  
  if  $i = n + 1$  then return ( $pa \leq W$ ) ?  $va$  :  $-\infty$ ;  
  return  $\text{máx}(\text{mochila}(i + 1, pa, va), \text{mochila}(i + 1, pa + w_i, va + v_i))$  ;
```

- La clase pasada dijimos: «mochila(1, 0, 0) devuelve el óptimo». ¿Por qué? Porque recorre todas las hojas y se queda con la mejor válida. Eso es una *intuición*, no una demostración.

El ejemplo de siempre: mochila

n objetos, pesos $w_j \in \mathbb{N}$, valores $v_j \in \mathbb{N}$, capacidad W . Sols = $\{0, 1\}^n$,
 $\text{válida}(a) := [\sum_j a_j w_j \leq W]$, $\text{valor}(a) := \sum_j a_j v_j$.

```
function mochila( $i$ ,  $pa$ ,  $va$ )  
  if  $i = n + 1$  then return  $(pa \leq W) ? va : -\infty$ ;  
  return  $\text{máx}(\text{mochila}(i + 1, pa, va), \text{mochila}(i + 1, pa + w_i, va + v_i))$  ;
```

- La clase pasada dijimos: «mochila(1, 0, 0) devuelve el óptimo». ¿Por qué? Porque recorre todas las hojas y se queda con la mejor válida. Eso es una *intuición*, no una demostración.
- Para demostrarlo hay que decir qué devuelve mochila(i, pa, va) **para cualquier** (i, pa, va), no sólo para (1, 0, 0). Eso es la **semántica**.

Semántica de una función recursiva

Definición (informal)

La **semántica** de una función $f(x_1, \dots, x_k)$ es una fórmula matemática $\sigma(x_1, \dots, x_k)$, definida **sin usar** f , tal que lo que queremos demostrar es:

para todo $(x_1, \dots, x_k)$ en el dominio, $f(x_1, \dots, x_k) = \sigma(x_1, \dots, x_k)$.

Semántica de una función recursiva

Definición (informal)

La **semántica** de una función $f(x_1, \dots, x_k)$ es una fórmula matemática $\sigma(x_1, \dots, x_k)$, definida **sin usar** f , tal que lo que queremos demostrar es:

para todo $(x_1, \dots, x_k)$ en el dominio, $f(x_1, \dots, x_k) = \sigma(x_1, \dots, x_k)$.

- Es la *especificación* de la función. Sin ella, «la función es correcta» no significa nada: ¿correcta respecto de qué?

Semántica de una función recursiva

Definición (informal)

La **semántica** de una función $f(x_1, \dots, x_k)$ es una fórmula matemática $\sigma(x_1, \dots, x_k)$, definida **sin usar** f , tal que lo que queremos demostrar es:

para todo $(x_1, \dots, x_k)$ en el dominio, $f(x_1, \dots, x_k) = \sigma(x_1, \dots, x_k)$.

- Es la *especificación* de la función. Sin ella, «la función es correcta» no significa nada: ¿correcta respecto de qué?
- Tiene que hablar de **todos** los parámetros y de **ningún** otro dato que no sea la entrada del problema. Si la función recibe (i, pa, va) , la semántica no puede decir «la parcial p » a menos que la defina a partir de (i, pa, va) .

Semántica de una función recursiva

Definición (informal)

La **semántica** de una función $f(x_1, \dots, x_k)$ es una fórmula matemática $\sigma(x_1, \dots, x_k)$, definida **sin usar** f , tal que lo que queremos demostrar es:

para todo $(x_1, \dots, x_k)$ en el dominio, $f(x_1, \dots, x_k) = \sigma(x_1, \dots, x_k)$.

- Es la *especificación* de la función. Sin ella, «la función es correcta» no significa nada: ¿correcta respecto de qué?
- Tiene que hablar de **todos** los parámetros y de **ningún** otro dato que no sea la entrada del problema. Si la función recibe (i, pa, va) , la semántica no puede decir «la parcial p » a menos que la defina a partir de (i, pa, va) .
- En backtracking casi siempre tiene la forma: «*el máximo (o: existe, o: la cantidad) sobre el conjunto de **completaciones** consistentes con los parámetros*».

Semánticas malas

Para $\text{mochila}(i, pa, va)$:

- «Devuelve el máximo valor de la mochila.» **Falso** para $(i, pa, va) \neq (1, 0, 0)$, y no menciona los parámetros.

Semánticas malas

Para $\text{mochila}(i, pa, va)$:

- «Devuelve el máximo valor de la mochila.» **Falso** para $(i, pa, va) \neq (1, 0, 0)$, y no menciona los parámetros.
- «Devuelve el mejor valor que se puede obtener desde esta llamada.» **Circular:** «desde esta llamada» sólo tiene sentido si ya sabemos qué hace la función.

Semánticas malas

Para mochila(i, pa, va):

- «Devuelve el máximo valor de la mochila.» **Falso** para $(i, pa, va) \neq (1, 0, 0)$, y no menciona los parámetros.
- «Devuelve el mejor valor que se puede obtener desde esta llamada.» **Circular:** «desde esta llamada» sólo tiene sentido si ya sabemos qué hace la función.
- «Devuelve el máximo valor de las extensiones válidas de p .» **Casi:** ¿qué es p ? La función no lo recibe. Hay que decir qué tienen en común todas las parciales con ese (i, pa, va) .

Semánticas malas

Para mochila(i, pa, va):

- «Devuelve el máximo valor de la mochila.» **Falso** para $(i, pa, va) \neq (1, 0, 0)$, y no menciona los parámetros.
- «Devuelve el mejor valor que se puede obtener desde esta llamada.» **Circular:** «desde esta llamada» sólo tiene sentido si ya sabemos qué hace la función.
- «Devuelve el máximo valor de las extensiones válidas de p .» **Casi:** ¿qué es p ? La función no lo recibe. Hay que decir qué tienen en común todas las parciales con ese (i, pa, va) .
- «Devuelve $\max(\text{mochila}(i + 1, \dots), \text{mochila}(i + 1, \dots))$.» **Es la definición**, no la semántica. Usa f para hablar de f .

Semánticas malas

Para mochila(i, pa, va):

- «Devuelve el máximo valor de la mochila.» **Falso** para $(i, pa, va) \neq (1, 0, 0)$, y no menciona los parámetros.
- «Devuelve el mejor valor que se puede obtener desde esta llamada.» **Circular:** «desde esta llamada» sólo tiene sentido si ya sabemos qué hace la función.
- «Devuelve el máximo valor de las extensiones válidas de p .» **Casi:** ¿qué es p ? La función no lo recibe. Hay que decir qué tienen en común todas las parciales con ese (i, pa, va) .
- «Devuelve $\max(\text{mochila}(i + 1, \dots), \text{mochila}(i + 1, \dots))$.» **Es la definición**, no la semántica. Usa f para hablar de f .

Prueba rápida: si en la semántica aparece el nombre de la función, o la palabra «llamada», o «el algoritmo», está mal.

Completaciones

Toda parcial con resumen (i, pa, va) tiene pendientes los objetos $i, \dots, n$; lo que falta es un vector $b = (b_i, \dots, b_n)$.

Completaciones

Toda parcial con resumen (i, pa, va) tiene pendientes los objetos $i, \dots, n$; lo que falta es un vector $b = (b_i, \dots, b_n)$.

Definición

Para $1 \leq i \leq n+1$ y $pa \in \mathbb{N}$, las **completaciones válidas** son

$$\text{Compl}(i, pa) := \left\{ b \in \{0, 1\}^{n-i+1} : pa + \sum_{j=i}^n b_j w_j \leq W \right\}.$$

(Para $i = n+1$ el único vector es el vacío $()$.)

Completaciones

Toda parcial con resumen (i, pa, va) tiene pendientes los objetos $i, \dots, n$; lo que falta es un vector $b = (b_i, \dots, b_n)$.

Definición

Para $1 \leq i \leq n+1$ y $pa \in \mathbb{N}$, las **completaciones válidas** son

$$\text{Compl}(i, pa) := \left\{ b \in \{0, 1\}^{n-i+1} : pa + \sum_{j=i}^n b_j w_j \leq W \right\}.$$

(Para $i = n+1$ el único vector es el vacío $()$.)

- $\text{Compl}(1, 0) = \text{Sols}_{\text{válidas}}$. $\text{Compl}(n+1, pa) = \{()\}$ si $pa \leq W$, y $= \emptyset$ si $pa > W$.

Completaciones

Toda parcial con resumen (i, pa, va) tiene pendientes los objetos $i, \dots, n$; lo que falta es un vector $b = (b_i, \dots, b_n)$.

Definición

Para $1 \leq i \leq n+1$ y $pa \in \mathbb{N}$, las **completaciones válidas** son

$$\text{Compl}(i, pa) := \left\{ b \in \{0, 1\}^{n-i+1} : pa + \sum_{j=i}^n b_j w_j \leq W \right\}.$$

(Para $i = n+1$ el único vector es el vacío $()$.)

- $\text{Compl}(1, 0) = \text{Sols}_{\text{válidas}}$. $\text{Compl}(n+1, pa) = \{()\}$ si $pa \leq W$, y $= \emptyset$ si $pa > W$.
- Para una parcial p con resumen (i, pa) :
 $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \{ p \oplus b : b \in \text{Compl}(i, pa) \}$. Es el puente entre «parciales» (la clase pasada) y «parámetros» (hoy).

La semántica

Semántica de mochila

Para todo $1 \leq i \leq n+1$ y $pa, va \in \mathbb{N}$:

$$M(i, pa, va) := \max \left\{ va + \sum_{j=i}^n b_j v_j : b \in \text{Compl}(i, pa) \right\}, \quad \text{con } \max \emptyset := -\infty.$$

Afirmación a demostrar: $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

La semántica

Semántica de mochila

Para todo $1 \leq i \leq n+1$ y $pa, va \in \mathbb{N}$:

$$M(i, pa, va) := \max \left\{ va + \sum_{j=i}^n b_j v_j : b \in \text{Compl}(i, pa) \right\}, \quad \text{con } \max \emptyset := -\infty.$$

Afirmación a demostrar: $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- En palabras: «el mayor valor total alcanzable eligiendo entre los objetos $i, \dots, n$, si ya se acumuló peso pa y valor va ; $-\infty$ si ninguna elección respeta la capacidad».

La semántica

Semántica de mochila

Para todo $1 \leq i \leq n + 1$ y $pa, va \in \mathbb{N}$:

$$M(i, pa, va) := \max \left\{ va + \sum_{j=i}^n b_j v_j : b \in \text{Compl}(i, pa) \right\}, \quad \text{con } \max \emptyset := -\infty.$$

Afirmación a demostrar: $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- En palabras: «el mayor valor total alcanzable eligiendo entre los objetos $i, \dots, n$, si ya se acumuló peso pa y valor va ; $-\infty$ si ninguna elección respeta la capacidad».
- Y el problema original: $M(1, 0, 0) = \max\{\text{valor}(a) : a \in \text{Sols}_{\text{válidas}}\}$, porque $\text{Compl}(1, 0) = \text{Sols}_{\text{válidas}}$ y $0 + \sum_j b_j v_j = \text{valor}(b)$.

La semántica

Semántica de mochila

Para todo $1 \leq i \leq n + 1$ y $pa, va \in \mathbb{N}$:

$$M(i, pa, va) := \max \left\{ va + \sum_{j=i}^n b_j v_j : b \in \text{Compl}(i, pa) \right\}, \quad \text{con } \max \emptyset := -\infty.$$

Afirmación a demostrar: $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- En palabras: «el mayor valor total alcanzable eligiendo entre los objetos $i, \dots, n$, si ya se acumuló peso pa y valor va ; $-\infty$ si ninguna elección respeta la capacidad».
- Y el problema original: $M(1, 0, 0) = \max\{\text{valor}(a) : a \in \text{Sols}_{\text{válidas}}\}$, porque $\text{Compl}(1, 0) = \text{Sols}_{\text{válidas}}$ y $0 + \sum_j b_j v_j = \text{valor}(b)$.
- La convención $\max \emptyset = -\infty$ no es un detalle: sin ella la semántica no está definida cuando $pa > W$.

Semánticas para las otras variantes

Con el mismo $\text{Compl}(i, pa)$:

- **Decisión** («¿hay un subconjunto de peso $\leq W$ y valor $\geq K$?»):
$$D(i, pa, va) := \left[\exists b \in \text{Compl}(i, pa) : va + \sum_{j \geq i} b_j v_j \geq K \right].$$

Semánticas para las otras variantes

Con el mismo $\text{Compl}(i, pa)$:

- **Decisión** («¿hay un subconjunto de peso $\leq W$ y valor $\geq K$?»):
$$D(i, pa, va) := \left[\exists b \in \text{Compl}(i, pa) : va + \sum_{j \geq i} b_j v_j \geq K \right].$$
- **Conteo** («¿cuántos subconjuntos tienen peso exactamente W ?»):
$$C(i, pa) := \left| \{ b \in \{0, 1\}^{n-i+1} : pa + \sum_{j \geq i} b_j w_j = W \} \right|.$$

Semánticas para las otras variantes

Con el mismo $\text{Compl}(i, pa)$:

- **Decisión** («¿hay un subconjunto de peso $\leq W$ y valor $\geq K$?»):
$$D(i, pa, va) := \left[\exists b \in \text{Compl}(i, pa) : va + \sum_{j \geq i} b_j v_j \geq K \right].$$
- **Conteo** («¿cuántos subconjuntos tienen peso exactamente W ?»):
$$C(i, pa) := \left| \{ b \in \{0, 1\}^{n-i+1} : pa + \sum_{j \geq i} b_j w_j = W \} \right|.$$
- **Construcción**: la semántica es la de optimización, más «y *mejorSol* es un $a \in \text{Sols}_{\text{válidas}}$ con $\text{valor}(a) = \text{mejor}$ ». Lo demostramos después, con las podas.

Semánticas para las otras variantes

Con el mismo $\text{Compl}(i, pa)$:

- **Decisión** («¿hay un subconjunto de peso $\leq W$ y valor $\geq K$?»):

$$D(i, pa, va) := \left[\exists b \in \text{Compl}(i, pa) : va + \sum_{j \geq i} b_j v_j \geq K \right].$$
- **Conteo** («¿cuántos subconjuntos tienen peso exactamente W ?»):

$$C(i, pa) := \left| \{ b \in \{0, 1\}^{n-i+1} : pa + \sum_{j \geq i} b_j w_j = W \} \right|.$$
- **Construcción**: la semántica es la de optimización, más «y *mejorSol* es un $a \in \text{Sols}_{\text{válidas}}$ con $\text{valor}(a) = \text{mejor}$ ». Lo demostramos después, con las podas.

Patrón

operador { *valor de la completación : completación consistente con los parámetros* },
 con el neutro del operador para el conjunto vacío ($-\infty$, False, 0).

¿En qué hacemos inducción?

Queremos probar $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

¿En qué hacemos inducción?

Queremos probar $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- La llamada recursiva va de i a $i + 1$: el índice **crece**. La inducción tiene que ser sobre algo que **decrece** hacia el caso base: $m := n - i + 1$, la cantidad de decisiones que faltan.

¿En qué hacemos inducción?

Queremos probar $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- La llamada recursiva va de i a $i + 1$: el índice **crece**. La inducción tiene que ser sobre algo que **decrece** hacia el caso base: $m := n - i + 1$, la cantidad de decisiones que faltan.
- Los otros dos parámetros cambian en las llamadas $(pa + w_i, va + v_i)$. La HI tiene que servir para *cualquier* valor de ellos: van **adentro** de $P(m)$ con un $\forall$.

¿En qué hacemos inducción?

Queremos probar $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- La llamada recursiva va de i a $i + 1$: el índice **crece**. La inducción tiene que ser sobre algo que **decrece** hacia el caso base: $m := n - i + 1$, la cantidad de decisiones que faltan.
- Los otros dos parámetros cambian en las llamadas $(pa + w_i, va + v_i)$. La HI tiene que servir para *cualquier* valor de ellos: van **adentro** de $P(m)$ con un $\forall$.

La propiedad

$$P(m) : \quad \forall pa, va \in \mathbb{N}, \quad \text{mochila}(n - m + 1, pa, va) = M(n - m + 1, pa, va).$$

Probamos $P(m)$ para todo $0 \leq m \leq n$ por inducción en m . Alcanza con inducción simple: la recursión sólo usa $m - 1$.

¿En qué hacemos inducción?

Queremos probar $\text{mochila}(i, pa, va) = M(i, pa, va)$ para todo (i, pa, va) .

- La llamada recursiva va de i a $i + 1$: el índice **crece**. La inducción tiene que ser sobre algo que **decrece** hacia el caso base: $m := n - i + 1$, la cantidad de decisiones que faltan.
- Los otros dos parámetros cambian en las llamadas $(pa + w_i, va + v_i)$. La HI tiene que servir para *cualquier* valor de ellos: van **adentro** de $P(m)$ con un $\forall$.

La propiedad

$$P(m) : \quad \forall pa, va \in \mathbb{N}, \quad \text{mochila}(n - m + 1, pa, va) = M(n - m + 1, pa, va).$$

Probamos $P(m)$ para todo $0 \leq m \leq n$ por inducción en m . Alcanza con inducción simple: la recursión sólo usa $m - 1$.

Con la convención de la práctica (sufijos: $f(i, \dots)$ llama a $f(i - 1, \dots)$), la inducción es directamente en i . Misma demostración.

Un lema que se usa siempre

Lema (máximo de una unión)

Sean $A, B \subseteq \mathbb{R}$ finitos, con la convención $\max \emptyset = -\infty$. Entonces

$$\max(A \cup B) = \max(\max A, \max B).$$

Un lema que se usa siempre

Lema (máximo de una unión)

Sean $A, B \subseteq \mathbb{R}$ finitos, con la convención $\text{máx } \emptyset = -\infty$. Entonces

$$\text{máx}(A \cup B) = \text{máx}(\text{máx } A, \text{máx } B).$$

- **Demostración:** si ambos son vacíos, ambos lados son $-\infty$. Si uno es vacío, digamos B , $A \cup B = A$ y $\text{máx}(\text{máx } A, -\infty) = \text{máx } A$. Si ninguno es vacío: $\text{máx } A$ y $\text{máx } B$ están en $A \cup B$, así que $\text{máx}(A \cup B) \geq \text{ambos}$; y todo elemento de $A \cup B$ está en A o en B , así que es $\leq \text{máx } A$ o $\leq \text{máx } B$. $\square$

Un lema que se usa siempre

Lema (máximo de una unión)

Sean $A, B \subseteq \mathbb{R}$ finitos, con la convención $\max \emptyset = -\infty$. Entonces

$$\max(A \cup B) = \max(\max A, \max B).$$

- **Demostración:** si ambos son vacíos, ambos lados son $-\infty$. Si uno es vacío, digamos B , $A \cup B = A$ y $\max(\max A, -\infty) = \max A$. Si ninguno es vacío: $\max A$ y $\max B$ están en $A \cup B$, así que $\max(A \cup B) \geq \text{ambos}$; y todo elemento de $A \cup B$ está en A o en B , así que es $\leq \max A$ o $\leq \max B$. $\square$
- Lo mismo vale para $\forall$ con False (existe en la unión sii existe en alguna), y para $|\cdot|$ con 0 **si la unión es disjunta**: $|A \cup B| = |A| + |B|$.

Un lema que se usa siempre

Lema (máximo de una unión)

Sean $A, B \subseteq \mathbb{R}$ finitos, con la convención $\max \emptyset = -\infty$. Entonces

$$\max(A \cup B) = \max(\max A, \max B).$$

- **Demostración:** si ambos son vacíos, ambos lados son $-\infty$. Si uno es vacío, digamos B , $A \cup B = A$ y $\max(\max A, -\infty) = \max A$. Si ninguno es vacío: $\max A$ y $\max B$ están en $A \cup B$, así que $\max(A \cup B) \geq \text{ambos}$; y todo elemento de $A \cup B$ está en A o en B , así que es $\leq \max A$ o $\leq \max B$. $\square$
- Lo mismo vale para $\forall$ con False (existe en la unión sii existe en alguna), y para $|\cdot|$ con 0 **si la unión es disjunta**: $|A \cup B| = |A| + |B|$.
- Y para «trasladar»: $\max\{c + x : x \in A\} = c + \max A$ (con $c + (-\infty) = -\infty$).

Un lema que se usa siempre

Lema (máximo de una unión)

Sean $A, B \subseteq \mathbb{R}$ finitos, con la convención $\max \emptyset = -\infty$. Entonces

$$\max(A \cup B) = \max(\max A, \max B).$$

- **Demostración:** si ambos son vacíos, ambos lados son $-\infty$. Si uno es vacío, digamos B , $A \cup B = A$ y $\max(\max A, -\infty) = \max A$. Si ninguno es vacío: $\max A$ y $\max B$ están en $A \cup B$, así que $\max(A \cup B) \geq \text{ambos}$; y todo elemento de $A \cup B$ está en A o en B , así que es $\leq \max A$ o $\leq \max B$. $\square$
- Lo mismo vale para $\vee$ con False (existe en la unión sii existe en alguna), y para $|\cdot|$ con 0 **si la unión es disjunta**: $|A \cup B| = |A| + |B|$.
- Y para «trasladar»: $\max\{c + x : x \in A\} = c + \max A$ (con $c + (-\infty) = -\infty$).

Estos lemas son la razón por la cual «combinar sucesoras con $\max / \vee / +$ » funciona. En la práctica se citan, no se demuestran.

El plan de la demostración

- 1 **Caso base** $m = 0$ ($i = n + 1$). $\text{Compl}(n + 1, pa)$ es $\{()\}$ o $\emptyset$ según $pa \leq W$. Comparar con el primer *if*.

El plan de la demostración

- 1 **Caso base** $m = 0$ ($i = n + 1$). $\text{Compl}(n + 1, pa)$ es $\{()\}$ o $\emptyset$ según $pa \leq W$. Comparar con el primer *if*.
- 2 **Paso inductivo.** Sea $m \geq 1$, $i = n - m + 1 \leq n$, y pa , va cualesquiera. Asumimos $P(m - 1)$.

El plan de la demostración

- 1 **Caso base** $m = 0$ ($i = n + 1$). $\text{Compl}(n + 1, pa)$ es $\{()\}$ o $\emptyset$ según $pa \leq W$. Comparar con el primer *if*.
- 2 **Paso inductivo.** Sea $m \geq 1$, $i = n - m + 1 \leq n$, y pa , va cualesquiera. Asumimos $P(m - 1)$.
- 3 **Partir el conjunto de completaciones según la decisión i :**

$$\text{Compl}(i, pa) = \underbrace{\{0 \oplus b' : b' \in \text{Compl}(i + 1, pa)\}}_{b_i=0} \cup \underbrace{\{1 \oplus b' : b' \in \text{Compl}(i + 1, pa + w_i)\}}_{b_i=1}$$

El plan de la demostración

- 1 **Caso base** $m = 0$ ($i = n + 1$). $\text{Compl}(n + 1, pa)$ es $\{()\}$ o $\emptyset$ según $pa \leq W$. Comparar con el primer *if*.
- 2 **Paso inductivo.** Sea $m \geq 1$, $i = n - m + 1 \leq n$, y pa, va cualesquiera. Asumimos $P(m - 1)$.

- 3 **Partir el conjunto de completaciones según la decisión i :**

$$\text{Compl}(i, pa) = \underbrace{\{0 \oplus b' : b' \in \text{Compl}(i + 1, pa)\}}_{b_i=0} \cup \underbrace{\{1 \oplus b' : b' \in \text{Compl}(i + 1, pa + w_i)\}}_{b_i=1}$$

- 4 **Ver qué vale cada parte:** en la primera, $va + \sum_{j \geq i} b_j v_j = va + \sum_{j > i} b'_j v_j$; en la segunda, $= (va + v_i) + \sum_{j > i} b'_j v_j$.

El plan de la demostración

- 1 **Caso base** $m = 0$ ($i = n + 1$). $\text{Compl}(n + 1, pa)$ es $\{()\}$ o $\emptyset$ según $pa \leq W$. Comparar con el primer *if*.
- 2 **Paso inductivo.** Sea $m \geq 1$, $i = n - m + 1 \leq n$, y pa, va cualesquiera. Asumimos $P(m - 1)$.
- 3 **Partir el conjunto de completaciones según la decisión i :**

$$\text{Compl}(i, pa) = \underbrace{\{0 \oplus b' : b' \in \text{Compl}(i + 1, pa)\}}_{b_i=0} \cup \underbrace{\{1 \oplus b' : b' \in \text{Compl}(i + 1, pa + w_i)\}}_{b_i=1}$$

- 4 **Ver qué vale cada parte:** en la primera, $va + \sum_{j \geq i} b_j v_j = va + \sum_{j > i} b'_j v_j$; en la segunda, $= (va + v_i) + \sum_{j > i} b'_j v_j$.
- 5 **Aplicar la HI** a cada parte (con (pa, va) y con $(pa + w_i, va + v_i)$) y el **lema** del máximo de la unión.

El plan de la demostración

- 1 **Caso base** $m = 0$ ($i = n + 1$). $\text{Compl}(n + 1, pa)$ es $\{()\}$ o $\emptyset$ según $pa \leq W$. Comparar con el primer *if*.
- 2 **Paso inductivo.** Sea $m \geq 1$, $i = n - m + 1 \leq n$, y pa, va cualesquiera. Asumimos $P(m - 1)$.
- 3 **Partir el conjunto de completaciones según la decisión i :**

$$\text{Compl}(i, pa) = \underbrace{\{0 \oplus b' : b' \in \text{Compl}(i + 1, pa)\}}_{b_i=0} \cup \underbrace{\{1 \oplus b' : b' \in \text{Compl}(i + 1, pa + w_i)\}}_{b_i=1}$$

- 4 **Ver qué vale cada parte:** en la primera, $va + \sum_{j \geq i} b_j v_j = va + \sum_{j > i} b'_j v_j$; en la segunda, $= (va + v_i) + \sum_{j > i} b'_j v_j$.
- 5 **Aplicar la HI** a cada parte (con (pa, va) y con $(pa + w_i, va + v_i)$) y el **lema** del máximo de la unión.

→ Demostración completa en el pizarrón.

Después del pizarrón: qué se usó y qué no

- La HI se usó **dos veces**, con distintos (pa, va) . Por eso el $\forall pa, va$ va adentro de $P(m)$: si $P(m)$ fuera «para *este* pa », la HI no serviría para $pa + w_i$.

Después del pizarrón: qué se usó y qué no

- La HI se usó **dos veces**, con distintos (pa, va) . Por eso el $\forall pa, va$ va adentro de $P(m)$: si $P(m)$ fuera «para *este* pa », la HI no serviría para $pa + w_i$.
- La partición de $\text{Compl}(i, pa)$ tiene que ser **exactamente** eso: las dos partes cubren todo (todo b tiene $b_i \in \{0, 1\}$) y son disjuntas (no hace falta para máx; sí para contar).

Después del pizarrón: qué se usó y qué no

- La HI se usó **dos veces**, con distintos (pa, va) . Por eso el $\forall pa, va$ va adentro de $P(m)$: si $P(m)$ fuera «para *este* pa », la HI no serviría para $pa + w_i$.
- La partición de $\text{Compl}(i, pa)$ tiene que ser **exactamente** eso: las dos partes cubren todo (todo b tiene $b_i \in \{0, 1\}$) y son disjuntas (no hace falta para máx; sí para contar).
- **No se usó** que $w_j \geq 0$ ni que $v_j \geq 0$. La recurrencia es correcta con pesos negativos. Es la *poda* $pa > W$ la que necesita pesos no negativos. Distinguir qué hipótesis usa cada parte es lo que separa una demostración de una intuición.

Después del pizarrón: qué se usó y qué no

- La HI se usó **dos veces**, con distintos (pa, va) . Por eso el $\forall pa, va$ va adentro de $P(m)$: si $P(m)$ fuera «para *este* pa », la HI no serviría para $pa + w_i$.
- La partición de $\text{Compl}(i, pa)$ tiene que ser **exactamente** eso: las dos partes cubren todo (todo b tiene $b_i \in \{0, 1\}$) y son disjuntas (no hace falta para máx; sí para contar).
- **No se usó** que $w_j \geq 0$ ni que $v_j \geq 0$. La recurrencia es correcta con pesos negativos. Es la *poda* $pa > W$ la que necesita pesos no negativos. Distinguir qué hipótesis usa cada parte es lo que separa una demostración de una intuición.
- Tampoco se dijo nada de «el algoritmo recorre el árbol»: la demostración habla de **conjuntos y valores**, no de ejecuciones. Eso es lo que la hace corta.

El molde del paso inductivo

Para **cualquier** función de backtracking $f(\text{params})$ con semántica $\text{op}\{\text{valor}(b) : b \in S(\text{params})\}$:

El molde del paso inductivo

Para **cualquier** función de backtracking $f(\text{params})$ con semántica $\text{op}\{\text{valor}(b) : b \in S(\text{params})\}$:

- 1 Escribir $S(\text{params})$ como **unión disjunta** de una parte por cada decisión posible d para la próxima coordenada: $S = \bigcup_d \{d \oplus b' : b' \in S(\text{params}_d)\}$, donde params_d son los parámetros de la llamada recursiva para la decisión d .

El molde del paso inductivo

Para **cualquier** función de backtracking $f(\text{params})$ con semántica $\text{op}\{\text{valor}(b) : b \in S(\text{params})\}$:

- 1 Escribir $S(\text{params})$ como **unión disjunta** de una parte por cada decisión posible d para la próxima coordenada: $S = \bigcup_d \{d \oplus b' : b' \in S(\text{params}_d)\}$, donde params_d son los parámetros de la llamada recursiva para la decisión d .
- 2 Verificar que $\text{valor}(d \oplus b') = c_d + \text{valor}'(b')$ (o $= \text{valor}'(b')$, o $\text{valor} \wedge \text{algo}$): el valor de la completación se descompone como «lo que aporta la decisión d » más «el valor de la sub-completación».

El molde del paso inductivo

Para **cualquier** función de backtracking $f(\text{params})$ con semántica $\text{op}\{\text{valor}(b) : b \in S(\text{params})\}$:

- 1 Escribir $S(\text{params})$ como **unión disjunta** de una parte por cada decisión posible d para la próxima coordenada: $S = \bigcup_d \{d \oplus b' : b' \in S(\text{params}_d)\}$, donde params_d son los parámetros de la llamada recursiva para la decisión d .
- 2 Verificar que $\text{valor}(d \oplus b') = c_d + \text{valor}'(b')$ (o $= \text{valor}'(b')$, o $\text{valor} \wedge \text{algo}$): el valor de la completación se descompone como «lo que aporta la decisión d » más «el valor de la sub-completación».
- 3 Aplicar la HI a cada $f(\text{params}_d)$ (hay que chequear que params_d está en el dominio y tiene medida menor).

El molde del paso inductivo

Para **cualquier** función de backtracking $f(\text{params})$ con semántica $\text{op}\{\text{valor}(b) : b \in S(\text{params})\}$:

- 1 Escribir $S(\text{params})$ como **unión disjunta** de una parte por cada decisión posible d para la próxima coordenada: $S = \bigcup_d \{d \oplus b' : b' \in S(\text{params}_d)\}$, donde params_d son los parámetros de la llamada recursiva para la decisión d .
- 2 Verificar que $\text{valor}(d \oplus b') = c_d + \text{valor}'(b')$ (o $= \text{valor}'(b')$, o $\text{valor} \wedge \text{algo}$): el valor de la completación se descompone como «lo que aporta la decisión d » más «el valor de la sub-completación».
- 3 Aplicar la HI a cada $f(\text{params}_d)$ (hay que chequear que params_d está en el dominio y tiene medida menor).
- 4 Aplicar el lema del operador sobre la unión.

El molde del paso inductivo

Para **cualquier** función de backtracking $f(\text{params})$ con semántica $\text{op}\{\text{valor}(b) : b \in S(\text{params})\}$:

- 1 Escribir $S(\text{params})$ como **unión disjunta** de una parte por cada decisión posible d para la próxima coordenada: $S = \bigcup_d \{d \oplus b' : b' \in S(\text{params}_d)\}$, donde params_d son los parámetros de la llamada recursiva para la decisión d .
- 2 Verificar que $\text{valor}(d \oplus b') = c_d + \text{valor}'(b')$ (o $= \text{valor}'(b')$, o $\text{valor} \wedge \text{algo}$): el valor de la completación se descompone como «lo que aporta la decisión d » más «el valor de la sub-completación».
- 3 Aplicar la HI a cada $f(\text{params}_d)$ (hay que chequear que params_d está en el dominio y tiene medida menor).
- 4 Aplicar el lema del operador sobre la unión.

Los pasos 1 y 2 son los que cambian de problema a problema. Los pasos 3 y 4 son siempre iguales. Si el paso 1 no sale, casi siempre es porque a los parámetros les falta información (el *resumen* es insuficiente).

Una poda por factibilidad es un caso base más

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
    if  $pa > W$  then return  $-\infty$                                 // poda;  
    if  $i = n + 1$  then return  $va$ ;  
    return  $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

Una poda por factibilidad es un caso base más

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
  if  $pa > W$  then return  $-\infty$                                 // poda;  
  if  $i = n + 1$  then return  $va$ ;  
  return  $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

- **Afirmación:** mochila_3 tiene la *misma* semántica M. No hace falta rehacer la demostración: sólo agregar un caso.

Una poda por factibilidad es un caso base más

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
  if  $pa > W$  then return  $-\infty$                                 // poda;  
  if  $i = n + 1$  then return  $va$ ;  
  return  $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

- **Afirmación:** mochila_3 tiene la *misma* semántica M . No hace falta rehacer la demostración: sólo agregar un caso.
- **Lema (la poda):** si $pa > W$ entonces $\text{Compl}(i, pa) = \emptyset$ y luego $M(i, pa, va) = -\infty$.
Demostración: si $b \in \text{Compl}(i, pa)$, entonces $W \geq pa + \sum_{j \geq i} b_j w_j \geq pa > W$ (usando $w_j \geq 0$), absurdo.

Una poda por factibilidad es un caso base más

```
function mochila3( $i$ ,  $pa$ ,  $va$ )  
  if  $pa > W$  then return  $-\infty$  // poda;  
  if  $i = n + 1$  then return  $va$ ;  
  return  $\text{máx}(\text{mochila}_3(i + 1, pa, va), \text{mochila}_3(i + 1, pa + w_i, va + v_i))$  ;
```

- **Afirmación:** mochila_3 tiene la *misma* semántica M . No hace falta rehacer la demostración: sólo agregar un caso.
- **Lema (la poda):** si $pa > W$ entonces $\text{Compl}(i, pa) = \emptyset$ y luego $M(i, pa, va) = -\infty$.
Demostración: si $b \in \text{Compl}(i, pa)$, entonces $W \geq pa + \sum_{j \geq i} b_j w_j \geq pa > W$ (usando $w_j \geq 0$), absurdo.
- Con el lema, el nuevo primer *if* devuelve exactamente $M(i, pa, va)$ en su caso. En los demás casos la función es idéntica a mochila y la demostración anterior sigue palabra por palabra (la HI ahora es sobre mochila_3 ; el caso base $i = n + 1$ sólo se alcanza con $pa \leq W$, donde $M = va$).

Dos formas de decir lo mismo

En términos de parciales (la clase pasada):

Si $\neg R(p)$, entonces $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset$.

Se demuestra tomando $a \in \text{Ext}(p)$ arbitraria y mostrando $\neg \text{válida}(a)$.

En términos de parámetros (hoy):

Si los parámetros cumplen la condición de corte, el conjunto de completaciones es vacío, y la semántica vale el neutro.

Se demuestra tomando $b \in \text{Compl}(\dots)$ y llegando a un absurdo.

Dos formas de decir lo mismo

En términos de parciales (la clase pasada):

Si $\neg R(p)$, entonces $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset$.

Se demuestra tomando $a \in \text{Ext}(p)$ arbitraria y mostrando $\neg \text{válida}(a)$.

En términos de parámetros (hoy):

Si los parámetros cumplen la condición de corte, el conjunto de completaciones es vacío, y la semántica vale el neutro.

Se demuestra tomando $b \in \text{Compl}(\dots)$ y llegando a un absurdo.

- Son la misma afirmación (por el puente $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = p \oplus \text{Compl}(i, pa)$). Cualquiera de las dos vale como demostración en la práctica.

Dos formas de decir lo mismo

En términos de parciales (la clase pasada):

Si $\neg R(p)$, entonces $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = \emptyset$.

Se demuestra tomando $a \in \text{Ext}(p)$ arbitraria y mostrando $\neg \text{válida}(a)$.

En términos de parámetros (hoy):

Si los parámetros cumplen la condición de corte, el conjunto de completaciones es vacío, y la semántica vale el neutro.

Se demuestra tomando $b \in \text{Compl}(\dots)$ y llegando a un absurdo.

- Son la misma afirmación (por el puente $\text{Ext}(p) \cap \text{Sols}_{\text{válidas}} = p \oplus \text{Compl}(i, pa)$). Cualquiera de las dos vale como demostración en la práctica.
- La segunda tiene una ventaja: se integra directamente en la demostración por inducción de la recurrencia, como un caso más. No hace falta un argumento separado sobre «el algoritmo con poda visita un subconjunto de los nodos».

El problema con la poda por optimalidad

```
procedure mochila4(i, pa, va)  
  if pa > W then return;  
  if va + Si ≤ mejor then return                                //  $S_i = \sum_{j \geq i} v_j$ ;  
  if i = n + 1 then mejor ← va; return;  
  mochila4(i + 1, pa + wi, va + vi);  mochila4(i + 1, pa, va) ;
```

El problema con la poda por optimalidad

```
procedure mochila4(i, pa, va)  
  if pa > W then return;  
  if va + Si ≤ mejor then return                                // Si =  $\sum_{j \geq i} v_j$ ;  
  if i = n + 1 then mejor ← va; return;  
  mochila4(i + 1, pa + wi, va + vi);  mochila4(i + 1, pa, va);
```

- Ya no devuelve nada: el resultado está en la variable global *mejor*, que **cambia** durante la ejecución. La afirmación « $f(\dots) = \sigma(\dots)$ » no aplica tal cual.

El problema con la poda por optimalidad

```
procedure mochila4(i, pa, va)  
  if pa > W then return;  
  if va + Si ≤ mejor then return                                // Si =  $\sum_{j \geq i} v_j$ ;  
  if i = n + 1 then mejor ← va; return;  
  mochila4(i + 1, pa + wi, va + vi);  mochila4(i + 1, pa, va);
```

- Ya no devuelve nada: el resultado está en la variable global *mejor*, que **cambia** durante la ejecución. La afirmación « $f(\dots) = \sigma(\dots)$ » no aplica tal cual.
- Lo que sí podemos decir es cómo cambia *mejor* al ejecutar una llamada: una relación entre *el valor antes* y *el valor después*. Es un **invariante**, como en los algoritmos iterativos.

Lo que hay que demostrar

Notación: para una llamada $\text{mochila}_4(i, pa, va)$, sean mejor_0 el valor de *mejor* justo antes y mejor_1 el valor justo después.

Lo que hay que demostrar

Notación: para una llamada $mochila_4(i, pa, va)$, sean $mejor_0$ el valor de *mejor* justo antes y $mejor_1$ el valor justo después.

Tres hechos

- (H1) *mejor* **nunca decrece**: $mejor_1 \geq mejor_0$. (Sólo se asigna en la hoja, cuando pasó el *if* $va + S_{n+1} \leq mejor$, o sea $va > mejor$.)
- (H2) *mejor* es siempre $-\infty$ o el valor de **alguna solución válida**: en la hoja se asigna $va = \text{valor}(p)$ con $pa \leq W$ (pasó la primera poda).
- (H3) **(El lema principal.)** Para toda llamada: $mejor_1 \geq M(i, pa, va)$.

Lo que hay que demostrar

Notación: para una llamada $mochila_4(i, pa, va)$, sean $mejor_0$ el valor de *mejor* justo antes y $mejor_1$ el valor justo después.

Tres hechos

- (H1) *mejor* **nunca decrece**: $mejor_1 \geq mejor_0$. (Sólo se asigna en la hoja, cuando pasó el *if* $va + S_{n+1} \leq mejor$, o sea $va > mejor$.)
- (H2) *mejor* es siempre $-\infty$ o el valor de **alguna solución válida**: en la hoja se asigna $va = \text{valor}(p)$ con $pa \leq W$ (pasó la primera poda).
- (H3) **(El lema principal.)** Para toda llamada: $mejor_1 \geq M(i, pa, va)$.

- **Conclusión:** tras $mochila_4(1, 0, 0)$ con $mejor_0 = -\infty$: por (H3), $mejor \geq M(1, 0, 0) = \text{OPT}$; por (H2), $mejor \leq \text{OPT}$ (es el valor de una válida, o $-\infty$, pero $\text{OPT} > -\infty$). Luego $mejor = \text{OPT}$.

El plan para (H3)

Inducción en $m = n - i + 1$; $P(m)$: *para todo pa , va y todo valor inicial $mejor_0$, tras la llamada vale $mejor_1 \geq M(i, pa, va)$* . Cuatro casos, según qué *if* se ejecuta:

El plan para (H3)

Inducción en $m = n - i + 1$; $P(m)$: *para todo pa , va y todo valor inicial $mejor_0$, tras la llamada vale $mejor_1 \geq M(i, pa, va)$* . Cuatro casos, según qué *if* se ejecuta:

❶ **Poda por factibilidad.** $M = -\infty$ (lema anterior). Trivial.

El plan para (H3)

Inducción en $m = n - i + 1$; $P(m)$: *para todo pa , va y todo valor inicial $mejor_0$, tras la llamada vale $mejor_1 \geq M(i, pa, va)$* . Cuatro casos, según qué *if* se ejecuta:

- 1 **Poda por factibilidad.** $M = -\infty$ (lema anterior). Trivial.
- 2 **Poda por optimalidad.** $va + S_i \leq mejor_0$. Por la *cota*:
 $M(i, pa, va) \leq va + S_i \leq mejor_0 = mejor_1$.

El plan para (H3)

Inducción en $m = n - i + 1$; $P(m)$: *para todo pa , va y todo valor inicial $mejor_0$, tras la llamada vale $mejor_1 \geq M(i, pa, va)$* . Cuatro casos, según qué *if* se ejecuta:

- 1 **Poda por factibilidad.** $M = -\infty$ (lema anterior). Trivial.
- 2 **Poda por optimalidad.** $va + S_i \leq mejor_0$. Por la *cota*:
 $M(i, pa, va) \leq va + S_i \leq mejor_0 = mejor_1$.
- 3 **Hoja** ($m = 0$, ambas podas no cortaron). $mejor_1 = va = M(n + 1, pa, va)$ pues $pa \leq W$.

El plan para (H3)

Inducción en $m = n - i + 1$; $P(m)$: para todo pa , va y todo valor inicial $mejor_0$, tras la llamada vale $mejor_1 \geq M(i, pa, va)$. Cuatro casos, según qué if se ejecuta:

- ❶ **Poda por factibilidad.** $M = -\infty$ (lema anterior). Trivial.
- ❷ **Poda por optimalidad.** $va + S_i \leq mejor_0$. Por la cota:
 $M(i, pa, va) \leq va + S_i \leq mejor_0 = mejor_1$.
- ❸ **Hoja** ($m = 0$, ambas podas no cortaron). $mejor_1 = va = M(n + 1, pa, va)$ pues $pa \leq W$.
- ❹ **Recursión.** Sea $mejor'$ el valor entre las dos llamadas. Por HI en la primera:
 $mejor' \geq M(i + 1, pa + w_i, va + v_i)$. Por HI en la segunda: $mejor_1 \geq M(i + 1, pa, va)$.
 Por (H1): $mejor_1 \geq mejor'$. Luego $mejor_1 \geq \max(\dots, \dots) = M(i, pa, va)$ (por la recurrencia ya demostrada para M).

El plan para (H3)

Inducción en $m = n - i + 1$; $P(m)$: para todo pa , va y todo valor inicial $mejor_0$, tras la llamada vale $mejor_1 \geq M(i, pa, va)$. Cuatro casos, según qué *if* se ejecuta:

- ❶ **Poda por factibilidad.** $M = -\infty$ (lema anterior). Trivial.
- ❷ **Poda por optimalidad.** $va + S_i \leq mejor_0$. Por la cota:
 $M(i, pa, va) \leq va + S_i \leq mejor_0 = mejor_1$.
- ❸ **Hoja** ($m = 0$, ambas podas no cortaron). $mejor_1 = va = M(n + 1, pa, va)$ pues $pa \leq W$.
- ❹ **Recursión.** Sea $mejor'$ el valor entre las dos llamadas. Por HI en la primera:
 $mejor' \geq M(i + 1, pa + w_i, va + v_i)$. Por HI en la segunda: $mejor_1 \geq M(i + 1, pa, va)$.
Por (H1): $mejor_1 \geq mejor'$. Luego $mejor_1 \geq \max(\dots, \dots) = M(i, pa, va)$ (por la recurrencia ya demostrada para M).

→ Demostración en el pizarrón.

Qué se usó

- **La cota** $va + S_i \geq M(i, pa, va)$, sólo en el caso 2. Es *lo único* que cambia para otra poda por optimalidad: demostrar que la nueva U es cota; el resto es idéntico.

Qué se usó

- **La cota** $va + S_i \geq M(i, pa, va)$, sólo en el caso 2. Es *lo único* que cambia para otra poda por optimalidad: demostrar que la nueva U es cota; el resto es idéntico.
- **La monotonía** (H1), en el caso 4: la segunda llamada no puede «deshacer» lo que logró la primera.

Qué se usó

- **La cota** $va + S_i \geq M(i, pa, va)$, sólo en el caso 2. Es *lo único* que cambia para otra poda por optimalidad: demostrar que la nueva U es cota; el resto es idéntico.
- **La monotonía** (H1), en el caso 4: la segunda llamada no puede «deshacer» lo que logró la primera.
- **La recurrencia de M** , en el caso 4. Por eso conviene demostrar primero la versión sin podas.

Qué se usó

- **La cota** $va + S_i \geq M(i, pa, va)$, sólo en el caso 2. Es *lo único* que cambia para otra poda por optimalidad: demostrar que la nueva U es cota; el resto es idéntico.
- **La monotonía** (H1), en el caso 4: la segunda llamada no puede «deshacer» lo que logró la primera.
- **La recurrencia de M** , en el caso 4. Por eso conviene demostrar primero la versión sin podas.
- **El $\forall mejor_0$** adentro de $P(m)$: la segunda llamada arranca con un *mejor* distinto del de la primera.

Qué se usó

- **La cota** $va + S_i \geq M(i, pa, va)$, sólo en el caso 2. Es *lo único* que cambia para otra poda por optimalidad: demostrar que la nueva U es cota; el resto es idéntico.
- **La monotonía** (H1), en el caso 4: la segunda llamada no puede «deshacer» lo que logró la primera.
- **La recurrencia de M** , en el caso 4. Por eso conviene demostrar primero la versión sin podas.
- **El $\forall mejor_0$** adentro de $P(m)$: la segunda llamada arranca con un *mejor* distinto del de la primera.

En la práctica

«Proponer una poda por optimalidad y mostrar que es correcta» = demostrar que U (o L) es cota, y citar: «*como U acota a toda extensión válida, podar cuando $U(p) \leq mejor$ no altera el máximo*».

Construcción: la solución acompaña al valor

En `mochila5` (con *sol* compartido y *mejorSol*), agregamos a los hechos:

- **(H4)** Al entrar a la llamada (i, pa, va) , $sol[1..i - 1]$ es una parcial con peso pa y valor va , y al salir es **la misma**. (Cada llamada escribe sólo $sol[i]$, y el padre sólo lee $[1..i - 1]$.)

Construcción: la solución acompaña al valor

En *mochila₅* (con *sol* compartido y *mejorSol*), agregamos a los hechos:

- **(H4)** Al entrar a la llamada (i, pa, va) , $sol[1..i - 1]$ es una parcial con peso pa y valor va , y al salir es **la misma**. (Cada llamada escribe sólo $sol[i]$, y el padre sólo lee $[1..i - 1]$.)
- **(H5)** Si $mejor \neq -\infty$, *mejorSol* es válida y $valor(mejorSol) = mejor$. (Se asignan juntos en la hoja, donde por (H4) *sol* es una candidata con peso $pa \leq W$ y valor va .)

Construcción: la solución acompaña al valor

En mochila₅ (con *sol* compartido y *mejorSol*), agregamos a los hechos:

- **(H4)** Al entrar a la llamada (i, pa, va) , $sol[1..i - 1]$ es una parcial con peso pa y valor va , y al salir es **la misma**. (Cada llamada escribe sólo $sol[i]$, y el padre sólo lee $[1..i - 1]$.)
- **(H5)** Si $mejor \neq -\infty$, *mejorSol* es válida y $valor(mejorSol) = mejor$. (Se asignan juntos en la hoja, donde por (H4) *sol* es una candidata con peso $pa \leq W$ y valor va .)

Con (H5) y $mejor = OPT$: *mejorSol* es óptima. En la práctica, (H4) se resume en una frase: «*sol* se restaura al volver, así que $sol[1..i - 1]$ es la parcial actual».

¿Descansamos 10 minutos?



Problema completo 1: de punta a punta

Problema: Conjunto independiente de tamaño k

Dado un grafo $G = (V, E)$ con $V = \{1, \dots, n\}$ y un entero $k \geq 0$, decidir si existe $I \subseteq V$ con $|I| = k$ tal que ningún par de vértices de I es adyacente.

Problema completo 1: de punta a punta

Problema: Conjunto independiente de tamaño k

Dado un grafo $G = (V, E)$ con $V = \{1, \dots, n\}$ y un entero $k \geq 0$, decidir si existe $I \subseteq V$ con $|I| = k$ tal que ningún par de vértices de I es adyacente.

- **Candidatas:** $\text{Sols} := \{0, 1\}^n$; a representa $I_a := \{v : a_v = 1\}$.

Problema completo 1: de punta a punta

Problema: Conjunto independiente de tamaño k

Dado un grafo $G = (V, E)$ con $V = \{1, \dots, n\}$ y un entero $k \geq 0$, decidir si existe $I \subseteq V$ con $|I| = k$ tal que ningún par de vértices de I es adyacente.

- **Candidatas:** $\text{Sols} := \{0, 1\}^n$; a representa $I_a := \{v : a_v = 1\}$.
- **Válidas:** $\text{válida}(a) := [\sum_v a_v = k] \wedge [\forall uv \in E : a_u = 0 \vee a_v = 0]$.

Problema completo 1: de punta a punta

Problema: Conjunto independiente de tamaño k

Dado un grafo $G = (V, E)$ con $V = \{1, \dots, n\}$ y un entero $k \geq 0$, decidir si existe $I \subseteq V$ con $|I| = k$ tal que ningún par de vértices de I es adyacente.

- **Candidatas:** $\text{Sols} := \{0, 1\}^n$; a representa $I_a := \{v : a_v = 1\}$.
- **Válidas:** $\text{válida}(a) := [\sum_v a_v = k] \wedge [\forall uv \in E : a_u = 0 \vee a_v = 0]$.
- **Pregunta:** decisión, $\exists a \in \text{Sols} : \text{válida}(a)$.

Problema completo 1: de punta a punta

Problema: Conjunto independiente de tamaño k

Dado un grafo $G = (V, E)$ con $V = \{1, \dots, n\}$ y un entero $k \geq 0$, decidir si existe $I \subseteq V$ con $|I| = k$ tal que ningún par de vértices de I es adyacente.

- **Candidatas:** $\text{Sols} := \{0, 1\}^n$; a representa $I_a := \{v : a_v = 1\}$.
- **Válidas:** $\text{válida}(a) := [\sum_v a_v = k] \wedge [\forall uv \in E : a_u = 0 \vee a_v = 0]$.
- **Pregunta:** decisión, $\exists a \in \text{Sols} : \text{válida}(a)$.
- **Parciales:** prefijos $p = (a_1, \dots, a_{i-1})$: ya decidimos qué vértices de $\{1, \dots, i-1\}$ entran. **Resumen:** el índice i , el conjunto $S := I_p$ de vértices ya elegidos (hace falta para saber si i es adyacente a alguno) y $r := k - |S|$, cuántos faltan.

La función recursiva y su semántica

Recurrencia

Para $1 \leq i \leq n+1$, $S \subseteq \{1, \dots, i-1\}$ independiente, $r \in \mathbb{Z}$; $N(i)$ son los vecinos de i :

$$\text{ind}(i, S, r) := \begin{cases} [r = 0] & \text{si } i = n+1, \\ ([N(i) \cap S = \emptyset] \wedge \text{ind}(i+1, S \cup \{i\}, r-1)) \vee \text{ind}(i+1, S, r) & \text{si } i \leq n. \end{cases}$$

Llamada inicial: $\text{ind}(1, \emptyset, k)$.

La función recursiva y su semántica

Recurrencia

Para $1 \leq i \leq n+1$, $S \subseteq \{1, \dots, i-1\}$ independiente, $r \in \mathbb{Z}$; $N(i)$ son los vecinos de i :

$$\text{ind}(i, S, r) := \begin{cases} [r = 0] & \text{si } i = n+1, \\ ([N(i) \cap S = \emptyset] \wedge \text{ind}(i+1, S \cup \{i\}, r-1)) \vee \text{ind}(i+1, S, r) & \text{si } i \leq n. \end{cases}$$

Llamada inicial: $\text{ind}(1, \emptyset, k)$.

Semántica

$$\text{ind}(i, S, r) = \left[\exists I \subseteq \{i, \dots, n\} : |I| = r \wedge S \cup I \text{ es independiente} \right].$$

La función recursiva y su semántica

Recurrencia

Para $1 \leq i \leq n+1$, $S \subseteq \{1, \dots, i-1\}$ independiente, $r \in \mathbb{Z}$; $N(i)$ son los vecinos de i :

$$\text{ind}(i, S, r) := \begin{cases} [r = 0] & \text{si } i = n+1, \\ ([N(i) \cap S = \emptyset] \wedge \text{ind}(i+1, S \cup \{i\}, r-1)) \vee \text{ind}(i+1, S, r) & \text{si } i \leq n. \end{cases}$$

Llamada inicial: $\text{ind}(1, \emptyset, k)$.

Semántica

$$\text{ind}(i, S, r) = \left[\exists I \subseteq \{i, \dots, n\} : |I| = r \wedge S \cup I \text{ es independiente} \right].$$

La semántica no habla de p , de «llamadas» ni del árbol: sólo de i , S , r y la entrada G . Con $r < 0$ o $r > n - i + 1$ el conjunto es vacío: podas a la vista.

La demostración: plan

Inducción en $m = n - i + 1$. $P(m)$: *para todo $S \subseteq \{1, \dots, n - m\}$ independiente y todo $r \in \mathbb{Z}$, $\text{ind}(n - m + 1, S, r)$ cumple su semántica.*

La demostración: plan

Inducción en $m = n - i + 1$. $P(m)$: *para todo $S \subseteq \{1, \dots, n - m\}$ independiente y todo $r \in \mathbb{Z}$, $\text{ind}(n - m + 1, S, r)$ cumple su semántica.*

- **Caso base** $m = 0$. Los $I \subseteq \emptyset$: sólo $I = \emptyset$, con $|I| = 0$, y $S \cup \emptyset = S$ es independiente por hipótesis. Existe uno con $|I| = r$ sii $r = 0$. Coincide con $[r = 0]$.

La demostración: plan

Inducción en $m = n - i + 1$. $P(m)$: para todo $S \subseteq \{1, \dots, n - m\}$ independiente y todo $r \in \mathbb{Z}$, $\text{ind}(n - m + 1, S, r)$ cumple su semántica.

- **Caso base** $m = 0$. Los $I \subseteq \emptyset$: sólo $I = \emptyset$, con $|I| = 0$, y $S \cup \emptyset = S$ es independiente por hipótesis. Existe uno con $|I| = r$ sii $r = 0$. Coincide con $[r = 0]$.
- **Paso.** Partir los $I \subseteq \{i, \dots, n\}$ según si $i \in I$:
 - $i \in I$: $I = \{i\} \cup I'$ con $I' \subseteq \{i + 1, \dots, n\}$, $|I'| = r - 1$, y $S \cup \{i\} \cup I'$ independiente. Esto último equivale a: $S \cup \{i\}$ independiente (o sea $N(i) \cap S = \emptyset$) **y** $(S \cup \{i\}) \cup I'$ independiente. Si $N(i) \cap S \neq \emptyset$, no hay ninguno; si $= \emptyset$, es la semántica de $\text{ind}(i + 1, S \cup \{i\}, r - 1)$, que por HI la recurrencia computa (notar: $S \cup \{i\}$ es independiente, así que está en el dominio de $P(m - 1)$).
 - $i \notin I$: $I \subseteq \{i + 1, \dots, n\}$, $|I| = r$, $S \cup I$ independiente: es la semántica de $\text{ind}(i + 1, S, r)$, por HI.

La demostración: plan

Inducción en $m = n - i + 1$. $P(m)$: para todo $S \subseteq \{1, \dots, n - m\}$ independiente y todo $r \in \mathbb{Z}$, $\text{ind}(n - m + 1, S, r)$ cumple su semántica.

- **Caso base** $m = 0$. Los $I \subseteq \emptyset$: sólo $I = \emptyset$, con $|I| = 0$, y $S \cup \emptyset = S$ es independiente por hipótesis. Existe uno con $|I| = r$ sii $r = 0$. Coincide con $[r = 0]$.
- **Paso.** Partir los $I \subseteq \{i, \dots, n\}$ según si $i \in I$:
 - $i \in I$: $I = \{i\} \cup I'$ con $I' \subseteq \{i + 1, \dots, n\}$, $|I'| = r - 1$, y $S \cup \{i\} \cup I'$ independiente. Esto último equivale a: $S \cup \{i\}$ independiente (o sea $N(i) \cap S = \emptyset$) **y** $(S \cup \{i\}) \cup I'$ independiente. Si $N(i) \cap S \neq \emptyset$, no hay ninguno; si $= \emptyset$, es la semántica de $\text{ind}(i + 1, S \cup \{i\}, r - 1)$, que por HI la recurrencia computa (notar: $S \cup \{i\}$ es independiente, así que está en el dominio de $P(m - 1)$).
 - $i \notin I$: $I \subseteq \{i + 1, \dots, n\}$, $|I| = r$, $S \cup I$ independiente: es la semántica de $\text{ind}(i + 1, S, r)$, por HI.
- «Existe en la unión» = «existe en alguna parte»: el $\vee$. $\square$

Podas, con demostración

- **Factibilidad 1:** si $r > n - i + 1$, devolver False. *Demostración:* todo $I \subseteq \{i, \dots, n\}$ tiene $|I| \leq n - i + 1 < r$; el conjunto de la semántica es vacío. $\square$

Podas, con demostración

- **Factibilidad 1:** si $r > n - i + 1$, devolver False. *Demostración:* todo $I \subseteq \{i, \dots, n\}$ tiene $|I| \leq n - i + 1 < r$; el conjunto de la semántica es vacío. $\square$
- **Factibilidad 2:** si $r < 0$, devolver False. *Demostración:* $|I| \geq 0 > r$. $\square$ (En realidad la recurrencia nunca llama con $r < 0$ si la anterior está; es inofensiva.)

Podas, con demostración

- **Factibilidad 1:** si $r > n - i + 1$, devolver False. *Demostración:* todo $I \subseteq \{i, \dots, n\}$ tiene $|I| \leq n - i + 1 < r$; el conjunto de la semántica es vacío. $\square$
- **Factibilidad 2:** si $r < 0$, devolver False. *Demostración:* $|I| \geq 0 > r$. $\square$ (En realidad la recurrencia nunca llama con $r < 0$ si la anterior está; es inofensiva.)
- **Atajo de decisión:** si $r = 0$, devolver True. *Demostración:* $I = \emptyset$ sirve, porque S es independiente. $\square$ Corta *hacia arriba*: ya sabemos que existe.

Podas, con demostración

- **Factibilidad 1:** si $r > n - i + 1$, devolver False. *Demostración:* todo $I \subseteq \{i, \dots, n\}$ tiene $|I| \leq n - i + 1 < r$; el conjunto de la semántica es vacío. $\square$
- **Factibilidad 2:** si $r < 0$, devolver False. *Demostración:* $|I| \geq 0 > r$. $\square$ (En realidad la recurrencia nunca llama con $r < 0$ si la anterior está; es inofensiva.)
- **Atajo de decisión:** si $r = 0$, devolver True. *Demostración:* $I = \emptyset$ sirve, porque S es independiente. $\square$ Corta *hacia arriba*: ya sabemos que existe.
- **Ya incorporada:** sólo elegimos i si $N(i) \cap S = \emptyset$. Es la poda « S independiente» escrita como restricción a las sucesoras, y es lo que mantiene la hipótesis « S independiente» del dominio de la semántica.

Podas, con demostración

- **Factibilidad 1:** si $r > n - i + 1$, devolver False. *Demostración:* todo $I \subseteq \{i, \dots, n\}$ tiene $|I| \leq n - i + 1 < r$; el conjunto de la semántica es vacío. $\square$
- **Factibilidad 2:** si $r < 0$, devolver False. *Demostración:* $|I| \geq 0 > r$. $\square$ (En realidad la recurrencia nunca llama con $r < 0$ si la anterior está; es inofensiva.)
- **Atajo de decisión:** si $r = 0$, devolver True. *Demostración:* $I = \emptyset$ sirve, porque S es independiente. $\square$ Corta *hacia arriba*: ya sabemos que existe.
- **Ya incorporada:** sólo elegimos i si $N(i) \cap S = \emptyset$. Es la poda « S independiente» escrita como restricción a las sucesoras, y es lo que mantiene la hipótesis « S independiente» del dominio de la semántica.

Las tres se demuestran mostrando que el conjunto de la semántica es vacío (o no vacío, para el atajo). Ninguna necesita hablar del árbol.

Complejidad

- **Nodos:** árbol binario de altura n : $O(2^n)$.

Complejidad

- **Nodos:** árbol binario de altura n : $O(2^n)$.
- **Trabajo por nodo:** calcular $N(i) \cap S = \emptyset$. Con S como arreglo booleano global (marcar al bajar, desmarcar al volver) y listas de adyacencia: $O(|N(i)|) = O(n)$. Mantener r : $O(1)$.

Complejidad

- **Nodos:** árbol binario de altura n : $O(2^n)$.
- **Trabajo por nodo:** calcular $N(i) \cap S = \emptyset$. Con S como arreglo booleano global (marcar al bajar, desmarcar al volver) y listas de adyacencia: $O(|N(i)|) = O(n)$. Mantener r : $O(1)$.
- **Tiempo:** $O(n \cdot 2^n)$. Más fino: $\sum_{\text{nodos}} |N(i)|$; como en cada nivel i hay a lo sumo 2^{i-1} nodos, $\sum_i 2^{i-1} |N(i)| \leq 2^n \cdot \max_i |N(i)|$. No mejora el peor caso.

Complejidad

- **Nodos:** árbol binario de altura n : $O(2^n)$.
- **Trabajo por nodo:** calcular $N(i) \cap S = \emptyset$. Con S como arreglo booleano global (marcar al bajar, desmarcar al volver) y listas de adyacencia: $O(|N(i)|) = O(n)$. Mantener r : $O(1)$.
- **Tiempo:** $O(n \cdot 2^n)$. Más fino: $\sum_{\text{nodos}} |N(i)|$; como en cada nivel i hay a lo sumo 2^{i-1} nodos, $\sum_i 2^{i-1} |N(i)| \leq 2^n \cdot \max_i |N(i)|$. No mejora el peor caso.
- **Espacio:** profundidad n , $O(1)$ por llamada, más el arreglo de S : $O(n)$. Si en cambio pasáramos S por copia: $O(n)$ por llamada, $O(n^2)$ vivos.

Complejidad

- **Nodos:** árbol binario de altura n : $O(2^n)$.
- **Trabajo por nodo:** calcular $N(i) \cap S = \emptyset$. Con S como arreglo booleano global (marcar al bajar, desmarcar al volver) y listas de adyacencia: $O(|N(i)|) = O(n)$. Mantener r : $O(1)$.
- **Tiempo:** $O(n \cdot 2^n)$. Más fino: $\sum_{\text{nodos}} |N(i)|$; como en cada nivel i hay a lo sumo 2^{i-1} nodos, $\sum_i 2^{i-1} |N(i)| \leq 2^n \cdot \max_i |N(i)|$. No mejora el peor caso.
- **Espacio:** profundidad n , $O(1)$ por llamada, más el arreglo de S : $O(n)$. Si en cambio pasáramos S por copia: $O(n)$ por llamada, $O(n^2)$ vivos.
- **¿Es $\Theta(n \cdot 2^n)$?** Para afirmarlo habría que exhibir una familia de instancias *sin* solución (si hay, el atajo corta) donde las podas casi no actúan. No es obvio, y no se pide: la cota superior $O(n \cdot 2^n)$ alcanza. No escriban Θ si sólo demostraron O .

Problema completo 2: cuando el universo no es $\{0, 1\}^n$

Problema: Cambio mínimo

Hay r tipos de monedas, de valores $c_1, \dots, c_r \geq 1$ (cantidad ilimitada de cada una), y un monto $m \in \mathbb{N}$. Encontrar la mínima cantidad de monedas que suman exactamente m , o informar que es imposible.

Problema completo 2: cuando el universo no es $\{0, 1\}^n$

Problema: Cambio mínimo

Hay r tipos de monedas, de valores $c_1, \dots, c_r \geq 1$ (cantidad ilimitada de cada una), y un monto $m \in \mathbb{N}$. Encontrar la mínima cantidad de monedas que suman exactamente m , o informar que es imposible.

- Una solución es una **secuencia de monedas** de longitud variable. ¿Cuál es Sols? Tiene que ser finito. Como $c_j \geq 1$, toda secuencia que suma m tiene a lo sumo m monedas:

$$\text{Sols} := \{s \in \{1, \dots, r\}^\ell : 0 \leq \ell \leq m\}, \quad \text{válida}(s) := \left[\sum_{t=1}^{\ell} c_{s_t} = m \right], \quad \text{costo}(s) := \ell.$$

Problema completo 2: cuando el universo no es $\{0, 1\}^n$

Problema: Cambio mínimo

Hay r tipos de monedas, de valores $c_1, \dots, c_r \geq 1$ (cantidad ilimitada de cada una), y un monto $m \in \mathbb{N}$. Encontrar la mínima cantidad de monedas que suman exactamente m , o informar que es imposible.

- Una solución es una **secuencia de monedas** de longitud variable. ¿Cuál es Sols? Tiene que ser finito. Como $c_j \geq 1$, toda secuencia que suma m tiene a lo sumo m monedas:

$$\text{Sols} := \{s \in \{1, \dots, r\}^\ell : 0 \leq \ell \leq m\}, \quad \text{válida}(s) := \left[\sum_{t=1}^{\ell} c_{s_t} = m \right], \quad \text{costo}(s) := \ell.$$

- $|\text{Sols}| = \sum_{\ell=0}^m r^\ell = O(r^m)$. Minimizar ℓ .

Problema completo 2: cuando el universo no es $\{0, 1\}^n$

Problema: Cambio mínimo

Hay r tipos de monedas, de valores $c_1, \dots, c_r \geq 1$ (cantidad ilimitada de cada una), y un monto $m \in \mathbb{N}$. Encontrar la mínima cantidad de monedas que suman exactamente m , o informar que es imposible.

- Una solución es una **secuencia de monedas** de longitud variable. ¿Cuál es Sols? Tiene que ser finito. Como $c_j \geq 1$, toda secuencia que suma m tiene a lo sumo m monedas:

$$\text{Sols} := \{s \in \{1, \dots, r\}^\ell : 0 \leq \ell \leq m\}, \quad \text{válida}(s) := \left[\sum_{t=1}^{\ell} c_{s_t} = m \right], \quad \text{costo}(s) := \ell.$$

- $|\text{Sols}| = \sum_{\ell=0}^m r^\ell = O(r^m)$. Minimizar ℓ .
- Parciales:** prefijos $(s_1, \dots, s_\ell)$ con suma $\leq m$. **Resumen:** el monto que falta, $m' := m - \sum_t c_{s_t}$ (y ℓ , si hay poda por optimalidad).

Recurrencia y semántica

Recurrencia

Para $m' \in \mathbb{Z}$, con $1 + \infty := \infty$:

$$\text{cambio}(m') := \begin{cases} 0 & \text{si } m' = 0, \\ \infty & \text{si } m' < 0, \\ 1 + \min_{1 \leq j \leq r} \text{cambio}(m' - c_j) & \text{si } m' > 0. \end{cases}$$

Recurrencia y semántica

Recurrencia

Para $m' \in \mathbb{Z}$, con $1 + \infty := \infty$:

$$\text{cambio}(m') := \begin{cases} 0 & \text{si } m' = 0, \\ \infty & \text{si } m' < 0, \\ 1 + \min_{1 \leq j \leq r} \text{cambio}(m' - c_j) & \text{si } m' > 0. \end{cases}$$

Semántica

$$\text{cambio}(m') = \min \left\{ \ell \in \mathbb{N} : \exists \mathbf{s} \in \{1, \dots, r\}^\ell, \sum_{t=1}^{\ell} c_{s_t} = m' \right\}, \quad \text{con } \min \emptyset := \infty.$$

Recurrencia y semántica

Recurrencia

Para $m' \in \mathbb{Z}$, con $1 + \infty := \infty$:

$$\text{cambio}(m') := \begin{cases} 0 & \text{si } m' = 0, \\ \infty & \text{si } m' < 0, \\ 1 + \min_{1 \leq j \leq r} \text{cambio}(m' - c_j) & \text{si } m' > 0. \end{cases}$$

Semántica

$$\text{cambio}(m') = \min \left\{ \ell \in \mathbb{N} : \exists \mathbf{s} \in \{1, \dots, r\}^\ell, \sum_{t=1}^{\ell} c_{s_t} = m' \right\}, \quad \text{con } \min \emptyset := \infty.$$

- Sólo **un** parámetro: $(1, 2)$ y $(2, 1)$ tienen el mismo resumen, el mismo subproblema, resuelto dos veces. (*Guardarlo es programación dinámica: la clase que viene.*) El caso $m' < 0$ es una poda escrita como caso base.

La demostración: inducción fuerte en el monto

- ¿En qué inducimos? La llamada va de m' a $m' - c_j$, con $c_j \geq 1$: **el monto decrece**, pero no de a uno. **Inducción fuerte en m'** , $P(m')$: «cambio(m') cumple su semántica».

La demostración: inducción fuerte en el monto

- ¿En qué inducimos? La llamada va de m' a $m' - c_j$, con $c_j \geq 1$: **el monto decrece**, pero no de a uno. **Inducción fuerte en m'** , $P(m')$: «cambio(m') cumple su semántica».
- **Casos base:** $m' < 0$: el conjunto es vacío (una suma de valores ≥ 1 es ≥ 0), $\text{mín } \emptyset = \infty$. ✓ $m' = 0$: la secuencia vacía ($\ell = 0$) suma 0; $\text{mín} = 0$. ✓

La demostración: inducción fuerte en el monto

- ¿En qué inducimos? La llamada va de m' a $m' - c_j$, con $c_j \geq 1$: **el monto decrece**, pero no de a uno. **Inducción fuerte en m'** , $P(m')$: «cambio(m') cumple su semántica».
- **Casos base:** $m' < 0$: el conjunto es vacío (una suma de valores ≥ 1 es ≥ 0), $\text{mín } \emptyset = \infty$. ✓ $m' = 0$: la secuencia vacía ($\ell = 0$) suma 0; $\text{mín} = 0$. ✓
- **Paso.** $m' > 0$; asumimos $P(m'')$ para todo $m'' < m'$. Una secuencia s con $\ell \geq 1$ que suma m' tiene una **primera** moneda $s_1 = j$, y el resto $s' = (s_2, \dots, s_\ell)$ suma $m' - c_j$ y tiene $\ell - 1$ monedas. ($\ell = 0$ es imposible porque $m' > 0$.)

La demostración: inducción fuerte en el monto

- ¿En qué inducimos? La llamada va de m' a $m' - c_j$, con $c_j \geq 1$: **el monto decrece**, pero no de a uno. **Inducción fuerte en m'** , $P(m')$: «cambio(m') cumple su semántica».
- **Casos base:** $m' < 0$: el conjunto es vacío (una suma de valores ≥ 1 es ≥ 0), mín $\emptyset = \infty$. ✓ $m' = 0$: la secuencia vacía ($\ell = 0$) suma 0; mín = 0. ✓
- **Paso.** $m' > 0$; asumimos $P(m'')$ para todo $m'' < m'$. Una secuencia s con $\ell \geq 1$ que suma m' tiene una **primera** moneda $s_1 = j$, y el resto $s' = (s_2, \dots, s_\ell)$ suma $m' - c_j$ y tiene $\ell - 1$ monedas. ($\ell = 0$ es imposible porque $m' > 0$.)
- Partimos el conjunto de la semántica según $j = s_1$:
 $\{\ell : \dots\} = \bigcup_j \{1 + \ell' : \exists s' \in \{1..r\}^{\ell'} \text{ que suma } m' - c_j\}$, y por HI (aplicable porque $m' - c_j < m'$, **aunque sea negativo**: por eso el caso base $m' < 0$ está) el mínimo de cada parte es $1 + \text{cambio}(m' - c_j)$. Lema del mínimo de la unión. □

La demostración: inducción fuerte en el monto

- ¿En qué inducimos? La llamada va de m' a $m' - c_j$, con $c_j \geq 1$: **el monto decrece**, pero no de a uno. **Inducción fuerte en m'** , $P(m')$: «cambio(m') cumple su semántica».
- **Casos base:** $m' < 0$: el conjunto es vacío (una suma de valores ≥ 1 es ≥ 0), mín $\emptyset = \infty$. ✓ $m' = 0$: la secuencia vacía ($\ell = 0$) suma 0; mín = 0. ✓
- **Paso.** $m' > 0$; asumimos $P(m'')$ para todo $m'' < m'$. Una secuencia s con $\ell \geq 1$ que suma m' tiene una **primera** moneda $s_1 = j$, y el resto $s' = (s_2, \dots, s_\ell)$ suma $m' - c_j$ y tiene $\ell - 1$ monedas. ($\ell = 0$ es imposible porque $m' > 0$.)
- Partimos el conjunto de la semántica según $j = s_1$:
 $\{\ell : \dots\} = \bigcup_j \{1 + \ell' : \exists s' \in \{1..r\}^{\ell'} \text{ que suma } m' - c_j\}$, y por HI (aplicable porque $m' - c_j < m'$, **aunque sea negativo**: por eso el caso base $m' < 0$ está) el mínimo de cada parte es $1 + \text{cambio}(m' - c_j)$. Lema del mínimo de la unión. □

→ Demostración completa en el pizarrón.

Poda por optimalidad: una cota inferior

Versión con estado global *mejor* y el largo acumulado ℓ como parámetro: $\text{cambio}_2(m', \ell)$ actualiza $\text{mejor} \leftarrow \min(\text{mejor}, \ell)$ al llegar a $m' = 0$.

Poda por optimalidad: una cota inferior

Versión con estado global *mejor* y el largo acumulado ℓ como parámetro: $\text{cambio}_2(m', \ell)$ actualiza $\text{mejor} \leftarrow \min(\text{mejor}, \ell)$ al llegar a $m' = 0$.

Cota

$L(m', \ell) := \ell + \lceil m' / c_{\max} \rceil$, con $c_{\max} := \max_j c_j$. Podar si $L(m', \ell) \geq \text{mejor}$.

Poda por optimalidad: una cota inferior

Versión con estado global *mejor* y el largo acumulado ℓ como parámetro: $\text{cambio}_2(m', \ell)$ actualiza $\text{mejor} \leftarrow \min(\text{mejor}, \ell)$ al llegar a $m' = 0$.

Cota

$L(m', \ell) := \ell + \lceil m' / c_{\max} \rceil$, con $c_{\max} := \max_j c_j$. Podar si $L(m', \ell) \geq \text{mejor}$.

- **Demostración de que es cota:** toda completación usa ℓ' monedas que suman m' ; cada una vale a lo sumo $c_{\max}$, luego $\ell' \cdot c_{\max} \geq m'$, luego $\ell' \geq m' / c_{\max}$, y como ℓ' es entero, $\ell' \geq \lceil m' / c_{\max} \rceil$. El costo total es $\ell + \ell' \geq L(m', \ell)$. $\square$

Poda por optimalidad: una cota inferior

Versión con estado global *mejor* y el largo acumulado ℓ como parámetro: $\text{cambio}_2(m', \ell)$ actualiza $\text{mejor} \leftarrow \min(\text{mejor}, \ell)$ al llegar a $m' = 0$.

Cota

$L(m', \ell) := \ell + \lceil m' / c_{\max} \rceil$, con $c_{\max} := \max_j c_j$. Podar si $L(m', \ell) \geq \text{mejor}$.

- **Demostración de que es cota:** toda completación usa ℓ' monedas que suman m' ; cada una vale a lo sumo $c_{\max}$, luego $\ell' \cdot c_{\max} \geq m'$, luego $\ell' \geq m' / c_{\max}$, y como ℓ' es entero, $\ell' \geq \lceil m' / c_{\max} \rceil$. El costo total es $\ell + \ell' \geq L(m', \ell)$. $\square$
- Es la misma estructura que los mínimos de fila en asignación: *relajar* (dejar usar «fracciones» de la moneda más grande) para acotar.

Poda por optimalidad: una cota inferior

Versión con estado global *mejor* y el largo acumulado ℓ como parámetro: $\text{cambio}_2(m', \ell)$ actualiza $\text{mejor} \leftarrow \min(\text{mejor}, \ell)$ al llegar a $m' = 0$.

Cota

$L(m', \ell) := \ell + \lceil m' / c_{\max} \rceil$, con $c_{\max} := \max_j c_j$. Podar si $L(m', \ell) \geq \text{mejor}$.

- **Demostración de que es cota:** toda completación usa ℓ' monedas que suman m' ; cada una vale a lo sumo $c_{\max}$, luego $\ell' \cdot c_{\max} \geq m'$, luego $\ell' \geq m' / c_{\max}$, y como ℓ' es entero, $\ell' \geq \lceil m' / c_{\max} \rceil$. El costo total es $\ell + \ell' \geq L(m', \ell)$. $\square$
- Es la misma estructura que los mínimos de fila en asignación: *relajar* (dejar usar «fracciones» de la moneda más grande) para acotar.
- La cota trivial $L_1 = \ell$ también sirve (con $\geq$: si ya usé tantas monedas como la mejor solución, no voy a mejorar). Poda menos.

Complejidad

- El árbol tiene ramificación r y altura a lo sumo m (cada nivel resta al menos 1): $O(r^m)$ nodos, $O(r)$ trabajo por nodo (el mín sobre j): $O(r^{m+1})$. Más fino: altura $\leq \lceil m/c_{\min} \rceil$.

Complejidad

- El árbol tiene ramificación r y altura a lo sumo m (cada nivel resta al menos 1): $O(r^m)$ nodos, $O(r)$ trabajo por nodo (el mín sobre j): $O(r^{m+1})$. Más fino: altura $\leq \lceil m/c_{\min} \rceil$.
- Como recurrencia: $T(m') \leq r \cdot T(m' - 1) + O(r)$ para $m' \geq 1$ (cada llamada hace r llamadas a montos $\leq m' - 1$, y T es creciente). Se resuelve como en la clase de repaso: $T(m') = O(r^{m'})$.

Complejidad

- El árbol tiene ramificación r y altura a lo sumo m (cada nivel resta al menos 1): $O(r^m)$ nodos, $O(r)$ trabajo por nodo (el mín sobre j): $O(r^{m+1})$. Más fino: altura $\leq \lceil m/c_{\min} \rceil$.
- Como recurrencia: $T(m') \leq r \cdot T(m' - 1) + O(r)$ para $m' \geq 1$ (cada llamada hace r llamadas a montos $\leq m' - 1$, y T es creciente). Se resuelve como en la clase de repaso: $T(m') = O(r^{m'})$.
- **Espacio:** profundidad $\leq m$, $O(1)$ por llamada: $O(m)$. Si construimos la solución con un vector compartido: $O(m)$ también.

Complejidad

- El árbol tiene ramificación r y altura a lo sumo m (cada nivel resta al menos 1): $O(r^m)$ nodos, $O(r)$ trabajo por nodo (el mín sobre j): $O(r^{m+1})$. Más fino: altura $\leq \lceil m/c_{\min} \rceil$.
- Como recurrencia: $T(m') \leq r \cdot T(m' - 1) + O(r)$ para $m' \geq 1$ (cada llamada hace r llamadas a montos $\leq m' - 1$, y T es creciente). Se resuelve como en la clase de repaso: $T(m') = O(r^{m'})$.
- **Espacio:** profundidad $\leq m$, $O(1)$ por llamada: $O(m)$. Si construimos la solución con un vector compartido: $O(m)$ también.
- **Contraste:** el problema tiene sólo $m + 1$ subproblemas distintos ($m' \in \{0, \dots, m\}$). El árbol tiene $O(r^m)$ nodos porque los repite. Programación dinámica: $O(m \cdot r)$.

Complejidad

- El árbol tiene ramificación r y altura a lo sumo m (cada nivel resta al menos 1): $O(r^m)$ nodos, $O(r)$ trabajo por nodo (el mín sobre j): $O(r^{m+1})$. Más fino: altura $\leq \lceil m/c_{\min} \rceil$.
- Como recurrencia: $T(m') \leq r \cdot T(m' - 1) + O(r)$ para $m' \geq 1$ (cada llamada hace r llamadas a montos $\leq m' - 1$, y T es creciente). Se resuelve como en la clase de repaso: $T(m') = O(r^{m'})$.
- **Espacio:** profundidad $\leq m$, $O(1)$ por llamada: $O(m)$. Si construimos la solución con un vector compartido: $O(m)$ también.
- **Contraste:** el problema tiene sólo $m + 1$ subproblemas distintos ($m' \in \{0, \dots, m\}$). El árbol tiene $O(r^m)$ nodos porque los repite. Programación dinámica: $O(m \cdot r)$.

Nota sobre « T es creciente»

Para escribir $T(m' - c_j) \leq T(m' - 1)$ hace falta que T sea monótona, y eso también se demuestra (por inducción, o acotando por el árbol completo). En la práctica alcanza con decirlo.

Tres formas de acotar el tiempo

- 1 **Contar nodos.** $T \leq (\# \text{nodos}) \times (\text{trabajo máximo por nodo})$. Nodos de un árbol de ramificación $\leq k$ y altura $\leq h$: $\frac{k^{h+1}-1}{k-1} = O(k^h)$ (inducción en h ; está en el pizarrón de la clase pasada).

Tres formas de acotar el tiempo

- 1 **Contar nodos.** $T \leq (\# \text{nodos}) \times (\text{trabajo máximo por nodo})$. Nodos de un árbol de ramificación $\leq k$ y altura $\leq h$: $\frac{k^{h+1}-1}{k-1} = O(k^h)$ (inducción en h ; está en el pizarrón de la clase pasada).
- 2 **Recurrencia.** $T(m) \leq k \cdot T(m-1) + c$ para el trabajo con m decisiones restantes. Se resuelve por inducción, como $T(n) = 2T(n-4)$ en la clase de repaso:
 $T(m) \leq c' \cdot k^m$.

Tres formas de acotar el tiempo

- 1 **Contar nodos.** $T \leq (\# \text{nodos}) \times (\text{trabajo máximo por nodo})$. Nodos de un árbol de ramificación $\leq k$ y altura $\leq h$: $\frac{k^{h+1}-1}{k-1} = O(k^h)$ (inducción en h ; está en el pizarrón de la clase pasada).
- 2 **Recurrencia.** $T(m) \leq k \cdot T(m-1) + c$ para el trabajo con m decisiones restantes. Se resuelve por inducción, como $T(n) = 2T(n-4)$ en la clase de repaso:
 $T(m) \leq c' \cdot k^m$.
- 3 **Sumar por niveles.** $\sum_{\text{nivel } d} (\# \text{nodos en } d) \times (\text{trabajo en el nivel } d)$. Útil cuando el trabajo depende del nivel (por ejemplo, chequear la reina i contra $i-1$ anteriores: $\sum_i n^i \cdot i$).

Tres formas de acotar el tiempo

- 1 **Contar nodos.** $T \leq (\# \text{nodos}) \times (\text{trabajo máximo por nodo})$. Nodos de un árbol de ramificación $\leq k$ y altura $\leq h$: $\frac{k^{h+1}-1}{k-1} = O(k^h)$ (inducción en h ; está en el pizarrón de la clase pasada).
 - 2 **Recurrencia.** $T(m) \leq k \cdot T(m-1) + c$ para el trabajo con m decisiones restantes. Se resuelve por inducción, como $T(n) = 2T(n-4)$ en la clase de repaso:
 $T(m) \leq c' \cdot k^m$.
 - 3 **Sumar por niveles.** $\sum_{\text{nivel } d} (\# \text{nodos en } d) \times (\text{trabajo en el nivel } d)$. Útil cuando el trabajo depende del nivel (por ejemplo, chequear la reina i contra $i-1$ anteriores: $\sum_i n^i \cdot i$).
- Las tres dan cotas **superiores**, y valen con o sin podas (el árbol podado es un subárbol).

Tres formas de acotar el tiempo

- 1 **Contar nodos.** $T \leq (\# \text{nodos}) \times (\text{trabajo máximo por nodo})$. Nodos de un árbol de ramificación $\leq k$ y altura $\leq h$: $\frac{k^{h+1}-1}{k-1} = O(k^h)$ (inducción en h ; está en el pizarrón de la clase pasada).
 - 2 **Recurrencia.** $T(m) \leq k \cdot T(m-1) + c$ para el trabajo con m decisiones restantes. Se resuelve por inducción, como $T(n) = 2T(n-4)$ en la clase de repaso:
 $T(m) \leq c' \cdot k^m$.
 - 3 **Sumar por niveles.** $\sum_{\text{nivel } d} (\# \text{nodos en } d) \times (\text{trabajo en el nivel } d)$. Útil cuando el trabajo depende del nivel (por ejemplo, chequear la reina i contra $i-1$ anteriores: $\sum_i n^i \cdot i$).
- Las tres dan cotas **superiores**, y valen con o sin podas (el árbol podado es un subárbol).
 - Para una cota **inferior** (Ω) hay que exhibir instancias: «para $W \geq \sum w_j$, ninguna poda corta y se visitan $2^{n+1} - 1$ nodos».

El trabajo por nodo es donde se esconden los errores

Operación	Ingenua	Bien hecha
Extender la parcial ($p \oplus d$)	copiar: $O(n)$	vector compartido, push/pop: $O(1)$
¿ i es adyacente a algún elegido?	recorrer S : $O(n)$	arreglo de marcas + lista de adyacencia: $O(N(i))$
¿La tarea j está usada?	buscar en la parcial: $O(n)$	arreglo de marcas: $O(1)$
Cota $va + \sum_{j \geq i} v_j$	sumar: $O(n)$	sufijos precalculados: $O(1)$
Chequear validez en la hoja	desde cero: $O(n)$ o $O(n^2)$	incremental, en cada nodo: $O(1)$ o $O(n)$

El trabajo por nodo es donde se esconden los errores

Operación	Ingenua	Bien hecha
Extender la parcial ($p \oplus d$)	copiar: $O(n)$	vector compartido, push/pop: $O(1)$
¿ i es adyacente a algún elegido?	recorrer S : $O(n)$	arreglo de marcas + lista de adyacencia: $O(N(i))$
¿La tarea j está usada?	buscar en la parcial: $O(n)$	arreglo de marcas: $O(1)$
Cota $va + \sum_{j \geq i} v_j$	sumar: $O(n)$	sufijos precalculados: $O(1)$
Chequear validez en la hoja	desde cero: $O(n)$ o $O(n^2)$	incremental, en cada nodo: $O(1)$ o $O(n)$

- Un factor n no cambia el 2^n , pero en la práctica sí importa, y en un ejercicio hay que decir **cuál** de las dos versiones se está analizando.

El trabajo por nodo es donde se esconden los errores

Operación	Ingenua	Bien hecha
Extender la parcial ($p \oplus d$)	copiar: $O(n)$	vector compartido, push/pop: $O(1)$
¿ i es adyacente a algún elegido?	recorrer S : $O(n)$	arreglo de marcas + lista de adyacencia: $O(N(i))$
¿La tarea j está usada?	buscar en la parcial: $O(n)$	arreglo de marcas: $O(1)$
Cota $va + \sum_{j \geq i} v_j$	sumar: $O(n)$	sufijos precalculados: $O(1)$
Chequear validez en la hoja	desde cero: $O(n)$ o $O(n^2)$	incremental, en cada nodo: $O(1)$ o $O(n)$

- Un factor n no cambia el 2^n , pero en la práctica sí importa, y en un ejercicio hay que decir **cuál** de las dos versiones se está analizando.
- Lo que se pide: « $O(f(n))$ nodos, $O(g(n))$ por nodo, total $O(f(n) \cdot g(n))$; espacio $O(\text{profundidad} \cdot \text{estado por llamada} + \text{globales})$ ». Con una oración que justifique cada uno.

Enumeración y conteo: la salida manda

- Si hay que **listar** todas las soluciones válidas, el tiempo es al menos el tamaño de la salida: $\Omega(n \cdot |\text{Sols}_{\text{válidas}}|)$. Ninguna poda lo evita.

Enumeración y conteo: la salida manda

- Si hay que **listar** todas las soluciones válidas, el tiempo es al menos el tamaño de la salida: $\Omega(n \cdot |\text{Sols}_{\text{válidas}}|)$. Ninguna poda lo evita.
- Para **contar** no hace falta escribirlas, pero sí visitar cada hoja válida (salvo que se cuente «de a bloques», que ya no es backtracking puro): $\Omega(|\text{Sols}_{\text{válidas}}|)$.

Enumeración y conteo: la salida manda

- Si hay que **listar** todas las soluciones válidas, el tiempo es al menos el tamaño de la salida: $\Omega(n \cdot |\text{Sols}_{\text{válidas}}|)$. Ninguna poda lo evita.
- Para **contar** no hace falta escribirlas, pero sí visitar cada hoja válida (salvo que se cuente «de a bloques», que ya no es backtracking puro): $\Omega(|\text{Sols}_{\text{válidas}}|)$.
- Por eso, en conteo y enumeración, la respuesta honesta es « $O(\text{nodos} \cdot \text{trabajo})$ en el peor caso; y $\Omega(|\text{Sols}_{\text{válidas}}|)$ porque hay que visitar cada solución».

Enumeración y conteo: la salida manda

- Si hay que **listar** todas las soluciones válidas, el tiempo es al menos el tamaño de la salida: $\Omega(n \cdot |\text{Sols}_{\text{válidas}}|)$. Ninguna poda lo evita.
- Para **contar** no hace falta escribirlas, pero sí visitar cada hoja válida (salvo que se cuente «de a bloques», que ya no es backtracking puro): $\Omega(|\text{Sols}_{\text{válidas}}|)$.
- Por eso, en conteo y enumeración, la respuesta honesta es « $O(\text{nodos} \cdot \text{trabajo})$ en el peor caso; y $\Omega(|\text{Sols}_{\text{válidas}}|)$ porque hay que visitar cada solución».

Ejemplo: contar cuadrados latinos de orden n con $\text{Sols} = (\text{permutaciones})^n$: $O(n \cdot (n!)^n)$ nodos; los de orden 11 son del orden de 10^{47} (y para orden 12 la cantidad exacta no se conoce), así que ningún backtracking los va a contar uno por uno.

Errores comunes (I): en la semántica

- 1 **No hay semántica.** «La función devuelve la respuesta.» ¿Para qué parámetros?
¿Qué respuesta? Sin semántica no hay nada que demostrar.

Errores comunes (I): en la semántica

- 1 **No hay semántica.** «La función devuelve la respuesta.» ¿Para qué parámetros? ¿Qué respuesta? Sin semántica no hay nada que demostrar.
- 2 **Semántica circular.** « $f(i, \dots)$ devuelve lo mejor que se puede hacer a partir de la llamada i .» Describe la ejecución, no un valor matemático.

Errores comunes (I): en la semántica

- 1 **No hay semántica.** «La función devuelve la respuesta.» ¿Para qué parámetros? ¿Qué respuesta? Sin semántica no hay nada que demostrar.
- 2 **Semántica circular.** « $f(i, \dots)$ devuelve lo mejor que se puede hacer a partir de la llamada i .» Describe la ejecución, no un valor matemático.
- 3 **Parámetros que no aparecen.** La función recibe (i, pa, va) y la semántica habla de «la parcial p ». Hay que definir el conjunto de completaciones en términos de (i, pa, va) , o decir explícitamente que p es *cualquier* parcial con ese resumen.

Errores comunes (I): en la semántica

- ❶ **No hay semántica.** «La función devuelve la respuesta.» ¿Para qué parámetros? ¿Qué respuesta? Sin semántica no hay nada que demostrar.
- ❷ **Semántica circular.** « $f(i, \dots)$ devuelve lo mejor que se puede hacer a partir de la llamada i .» Describe la ejecución, no un valor matemático.
- ❸ **Parámetros que no aparecen.** La función recibe (i, pa, va) y la semántica habla de «la parcial p ». Hay que definir el conjunto de completaciones en términos de (i, pa, va) , o decir explícitamente que p es *cualquier* parcial con ese resumen.
- ❹ **Semántica sólo para la llamada inicial.** « $f(1, 0, 0)$ es el óptimo.» Es la conclusión, no la propiedad a inducir. La HI necesita la afirmación para *todos* los parámetros.

Errores comunes (I): en la semántica

- ❶ **No hay semántica.** «La función devuelve la respuesta.» ¿Para qué parámetros? ¿Qué respuesta? Sin semántica no hay nada que demostrar.
- ❷ **Semántica circular.** « $f(i, \dots)$ devuelve lo mejor que se puede hacer a partir de la llamada i .» Describe la ejecución, no un valor matemático.
- ❸ **Parámetros que no aparecen.** La función recibe (i, pa, va) y la semántica habla de «la parcial p ». Hay que definir el conjunto de completaciones en términos de (i, pa, va) , o decir explícitamente que p es *cualquier* parcial con ese resumen.
- ❹ **Semántica sólo para la llamada inicial.** « $f(1, 0, 0)$ es el óptimo.» Es la conclusión, no la propiedad a inducir. La HI necesita la afirmación para *todos* los parámetros.
- ❺ **Olvidar el conjunto vacío.** $\max \emptyset$, $\min \emptyset$: si no se fija la convención $(-\infty, \infty)$, la semántica está indefinida justo en los casos que las podas cortan.

Errores comunes (II): en la inducción

- 6 **Inducir en algo que no decrece.** «Inducción en i » cuando la llamada es a $i + 1$. Hay que inducir en $n - i + 1$, o cambiar la convención a sufijos.

Errores comunes (II): en la inducción

- 6 **Inducir en algo que no decrece.** «Inducción en i » cuando la llamada es a $i + 1$. Hay que inducir en $n - i + 1$, o cambiar la convención a sufijos.
- 7 **Inducción «en las llamadas»** o «en el árbol». Se puede hacer bien (inducción en la altura del subárbol), pero casi siempre se hace mal: la propiedad termina hablando del algoritmo en vez de los conjuntos.

Errores comunes (II): en la inducción

- 6 **Inducir en algo que no decrece.** «Inducción en i » cuando la llamada es a $i + 1$. Hay que inducir en $n - i + 1$, o cambiar la convención a sufijos.
- 7 **Inducción «en las llamadas»** o «en el árbol». Se puede hacer bien (inducción en la altura del subárbol), pero casi siempre se hace mal: la propiedad termina hablando del algoritmo en vez de los conjuntos.
- 8 **Usar la HI fuera del dominio.** cambio($m' - c_j$) con $m' - c_j < 0$; $\text{ind}(i + 1, S \cup \{i\}, \dots)$ sin verificar que $S \cup \{i\}$ es independiente. Chequear que los parámetros de cada llamada recursiva cumplen las hipótesis de P .

Errores comunes (II): en la inducción

- 6 **Inducir en algo que no decrece.** «Inducción en i » cuando la llamada es a $i + 1$. Hay que inducir en $n - i + 1$, o cambiar la convención a sufijos.
- 7 **Inducción «en las llamadas»** o «en el árbol». Se puede hacer bien (inducción en la altura del subárbol), pero casi siempre se hace mal: la propiedad termina hablando del algoritmo en vez de los conjuntos.
- 8 **Usar la HI fuera del dominio.** cambio($m' - c_j$) con $m' - c_j < 0$; $\text{ind}(i + 1, S \cup \{i\}, \dots)$ sin verificar que $S \cup \{i\}$ es independiente. Chequear que los parámetros de cada llamada recursiva cumplen las hipótesis de P .
- 9 **Partición que no lo es.** «Los subconjuntos que contienen a i y los que no» está bien; «los que usan la moneda j » (para algún j) no cubre todo ni es disjunto. Para máx y $\forall$ alcanza con cubrir; para contar hace falta disjunto.

Errores comunes (II): en la inducción

- 6 **Inducir en algo que no decrece.** «Inducción en i » cuando la llamada es a $i + 1$. Hay que inducir en $n - i + 1$, o cambiar la convención a sufijos.
- 7 **Inducción «en las llamadas»** o «en el árbol». Se puede hacer bien (inducción en la altura del subárbol), pero casi siempre se hace mal: la propiedad termina hablando del algoritmo en vez de los conjuntos.
- 8 **Usar la HI fuera del dominio.** cambio($m' - c_j$) con $m' - c_j < 0$; $\text{ind}(i + 1, S \cup \{i\}, \dots)$ sin verificar que $S \cup \{i\}$ es independiente. Chequear que los parámetros de cada llamada recursiva cumplen las hipótesis de P .
- 9 **Partición que no lo es.** «Los subconjuntos que contienen a i y los que no» está bien; «los que usan la moneda j » (para algún j) no cubre todo ni es disjunto. Para máx y $\forall$ alcanza con cubrir; para contar hace falta disjunto.
- 10 **Demostrar el código.** Razonar sobre variables, *ifs* y *returns* en vez de sobre la fórmula. Se acepta, pero es más largo y más fácil de errar. Escriban la fórmula y demuestren eso.

Errores comunes (III): en podas y complejidad

- 11 «**La poda es obvia**». La demostración de una poda es una oración: «sea a una extensión de p ; como $w_j \geq 0$, $\text{peso}(a) \geq pa > W$ ». Si no la escriben, no demostraron nada.

Errores comunes (III): en podas y complejidad

- 11 «**La poda es obvia**». La demostración de una poda es una oración: «sea a una extensión de p ; como $w_j \geq 0$, $\text{peso}(a) \geq pa > W$ ». Si no la escriben, no demostraron nada.
- 12 **Podar por optimalidad con $\geq$ cuando piden todas las óptimas**, o cortar al encontrar la primera válida cuando piden contar.

Errores comunes (III): en podas y complejidad

- 11 «**La poda es obvia**». La demostración de una poda es una oración: «sea a una extensión de p ; como $w_j \geq 0$, $\text{peso}(a) \geq pa > W$ ». Si no la escriben, no demostraron nada.
- 12 **Podar por optimalidad con $\geq$ cuando piden todas las óptimas**, o cortar al encontrar la primera válida cuando piden contar.
- 13 **Confundir el árbol de llamadas con el conjunto de válidas**. Contar hojas del árbol como si fueran soluciones (muchas son inválidas), o contar nodos del árbol *podado* de una instancia como si fuera el peor caso.

Errores comunes (III): en podas y complejidad

- 11 «**La poda es obvia**». La demostración de una poda es una oración: «sea a una extensión de p ; como $w_j \geq 0$, $\text{peso}(a) \geq pa > W$ ». Si no la escriben, no demostraron nada.
- 12 **Podar por optimalidad con $\geq$ cuando piden todas las óptimas**, o cortar al encontrar la primera válida cuando piden contar.
- 13 **Confundir el árbol de llamadas con el conjunto de válidas**. Contar hojas del árbol como si fueran soluciones (muchas son inválidas), o contar nodos del árbol *podado* de una instancia como si fuera el peor caso.
- 14 **Olvidar el trabajo por nodo**. « $O(2^n)$ » cuando cada nodo copia un vector de longitud n .

Errores comunes (III): en podas y complejidad

- 11 «**La poda es obvia**». La demostración de una poda es una oración: «sea a una extensión de p ; como $w_j \geq 0$, $\text{peso}(a) \geq pa > W$ ». Si no la escriben, no demostraron nada.
- 12 **Podar por optimalidad con $\geq$ cuando piden todas las óptimas**, o cortar al encontrar la primera válida cuando piden contar.
- 13 **Confundir el árbol de llamadas con el conjunto de válidas**. Contar hojas del árbol como si fueran soluciones (muchas son inválidas), o contar nodos del árbol *podado* de una instancia como si fuera el peor caso.
- 14 **Olvidar el trabajo por nodo**. « $O(2^n)$ » cuando cada nodo copia un vector de longitud n .
- 15 **Prometer Θ habiendo demostrado O** . La cota superior se obtiene del árbol completo; la inferior necesita una familia de instancias.

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.
- 2 Escriban la **semántica** como un conjunto (de completaciones) y un operador (máx, $\exists$, $|\cdot|$), con el neutro para el vacío. Sin usar el nombre de la función.

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.
- 2 Escriban la **semántica** como un conjunto (de completaciones) y un operador (máx, $\exists$, $|\cdot|$), con el neutro para el vacío. Sin usar el nombre de la función.
- 3 Induzcan en **lo que decrece**; los demás parámetros van con $\forall$ adentro de P .

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.
- 2 Escriban la **semántica** como un conjunto (de completaciones) y un operador (máx, $\exists$, $|\cdot|$), con el neutro para el vacío. Sin usar el nombre de la función.
- 3 Induzcan en **lo que decrece**; los demás parámetros van con $\forall$ adentro de P .
- 4 El paso inductivo es siempre: **partir** el conjunto según la próxima decisión, reconocer cada parte como el conjunto de una llamada recursiva, aplicar HI, aplicar el lema del operador.

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.
- 2 Escriban la **semántica** como un conjunto (de completaciones) y un operador (máx, $\exists$, $|\cdot|$), con el neutro para el vacío. Sin usar el nombre de la función.
- 3 Induzcan en **lo que decrece**; los demás parámetros van con $\forall$ adentro de P .
- 4 El paso inductivo es siempre: **partir** el conjunto según la próxima decisión, reconocer cada parte como el conjunto de una llamada recursiva, aplicar HI, aplicar el lema del operador.
- 5 Una poda por factibilidad es un lema «el conjunto es vacío». Una poda por optimalidad es un lema « U es cota». Nada más (y nada menos).

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.
- 2 Escriban la **semántica** como un conjunto (de completaciones) y un operador (máx, $\exists$, $|\cdot|$), con el neutro para el vacío. Sin usar el nombre de la función.
- 3 Induzcan en **lo que decrece**; los demás parámetros van con $\forall$ adentro de P .
- 4 El paso inductivo es siempre: **partir** el conjunto según la próxima decisión, reconocer cada parte como el conjunto de una llamada recursiva, aplicar HI, aplicar el lema del operador.
- 5 Una poda por factibilidad es un lema «el conjunto es vacío». Una poda por optimalidad es un lema « U es cota». Nada más (y nada menos).
- 6 Digan **qué hipótesis de la entrada** usa cada lema ($w_j \geq 0$, $c_j \geq 1$). Si no encuentran ninguna, sospechen.

Tips para demostrar algoritmos de fuerza bruta

- 1 Escriban la **fórmula** recursiva antes que el pseudocódigo. Se demuestra la fórmula.
- 2 Escriban la **semántica** como un conjunto (de completaciones) y un operador (máx, $\exists$, $|\cdot|$), con el neutro para el vacío. Sin usar el nombre de la función.
- 3 Induzcan en **lo que decrece**; los demás parámetros van con $\forall$ adentro de P .
- 4 El paso inductivo es siempre: **partir** el conjunto según la próxima decisión, reconocer cada parte como el conjunto de una llamada recursiva, aplicar HI, aplicar el lema del operador.
- 5 Una poda por factibilidad es un lema «el conjunto es vacío». Una poda por optimalidad es un lema « U es cota». Nada más (y nada menos).
- 6 Digan **qué hipótesis de la entrada** usa cada lema ($w_j \geq 0$, $c_j \geq 1$). Si no encuentran ninguna, sospechen.
- 7 Complejidad: nodos $\times$ trabajo por nodo, justificando ambos; O , no Θ , salvo que tengan la familia de instancias.